

Machine Learning for Biology and Health

CSCI 1851
Spring 2026

Ritambhara Singh

April 21, 2026
Tuesday



Today's goal – continue learning about Transformers

(1) Encoder: Positional encodings, Multi-headed attention, Modifications for efficiency

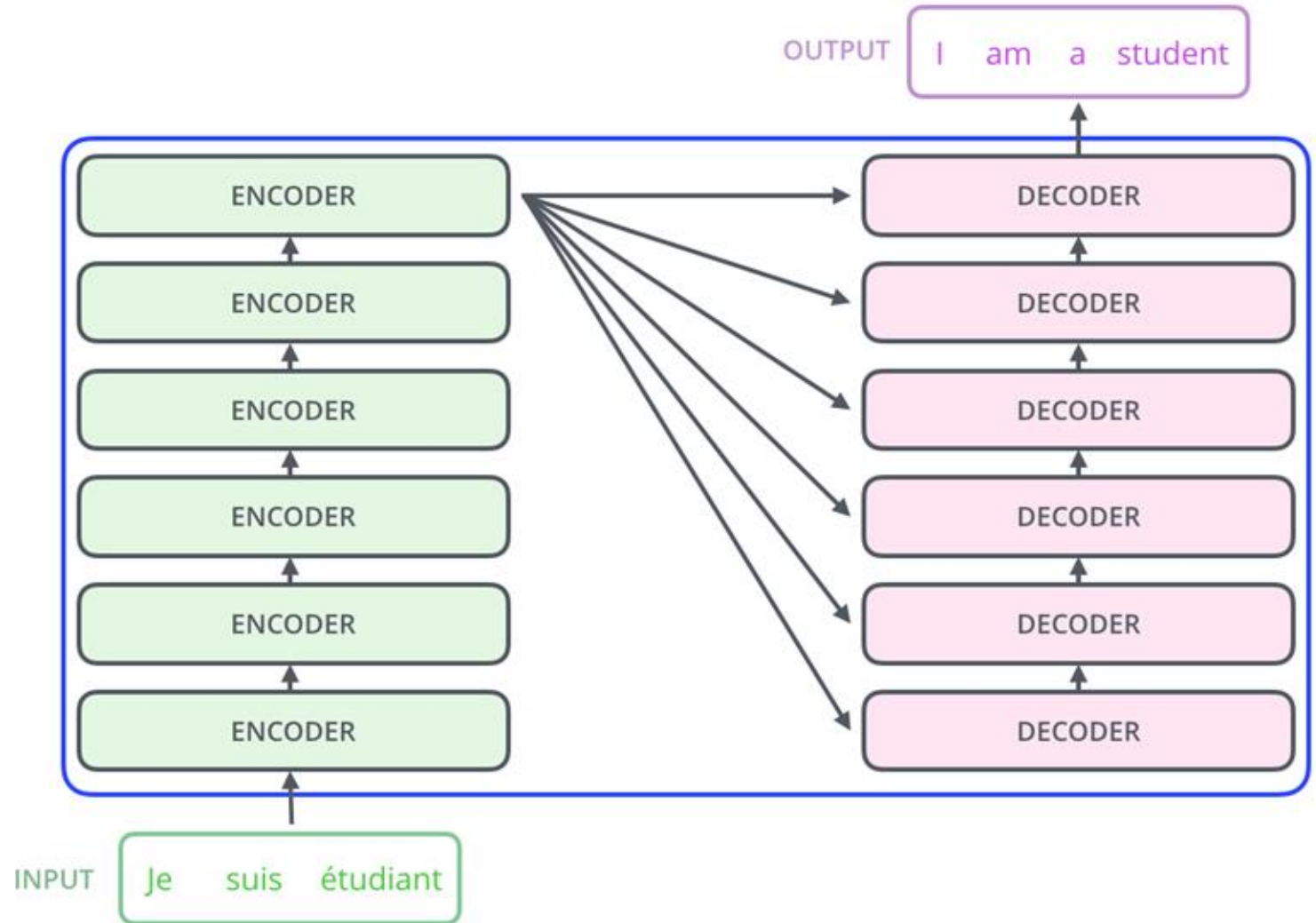
(2) Decoder

(3) Class activity: Implementing attention

(4) Foundation models

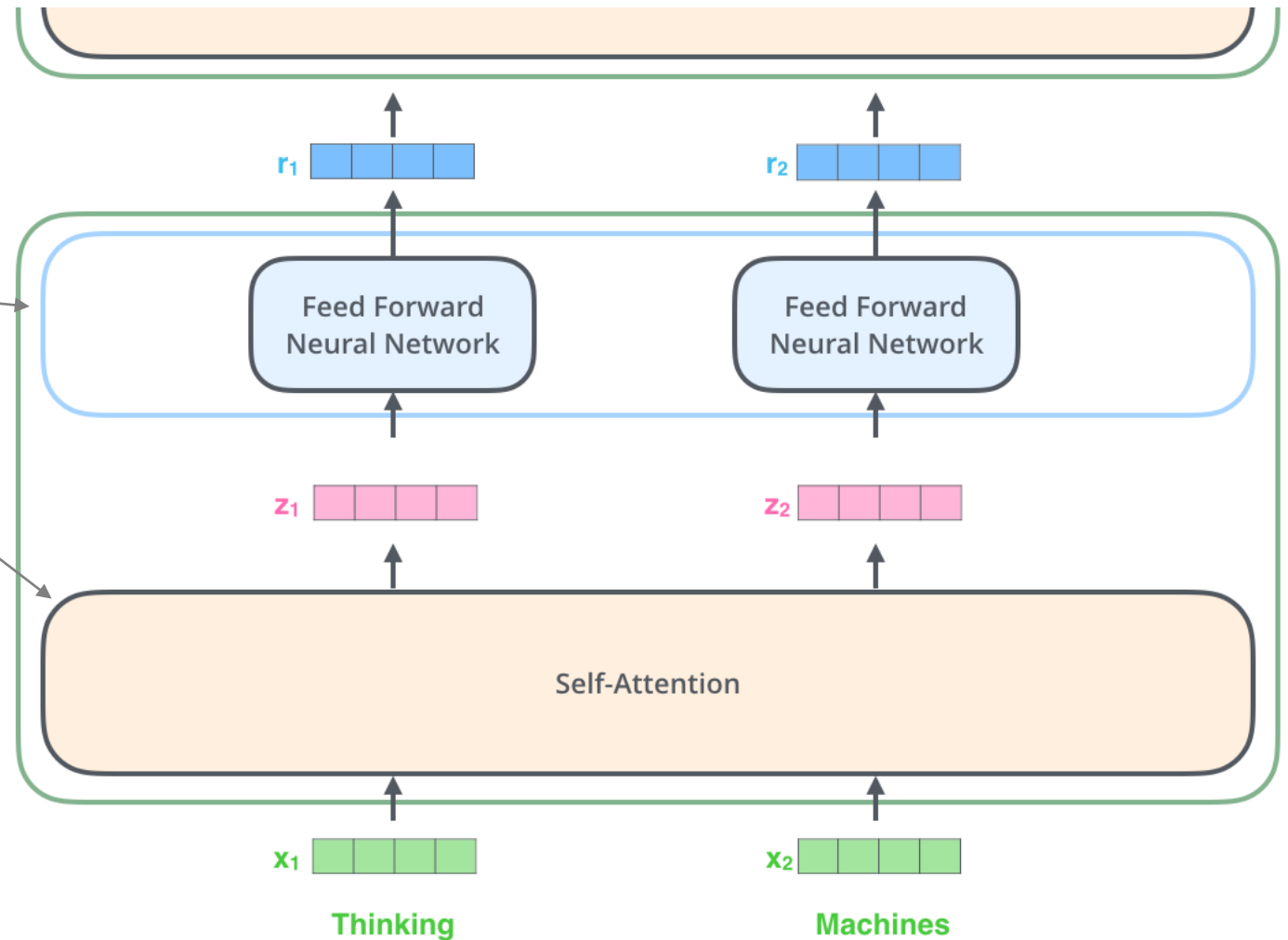
Transformer Model Overview

- The Transformer model breaks down into Encoder and Decoder blocks.
- At a high level, similar to the seq2seq architecture we've seen already...
- ...but there are no recurrent nets inside the Encoder and Decoder blocks!
- For better performance, often stack multiple Encoder and Decoder blocks (deeper network)



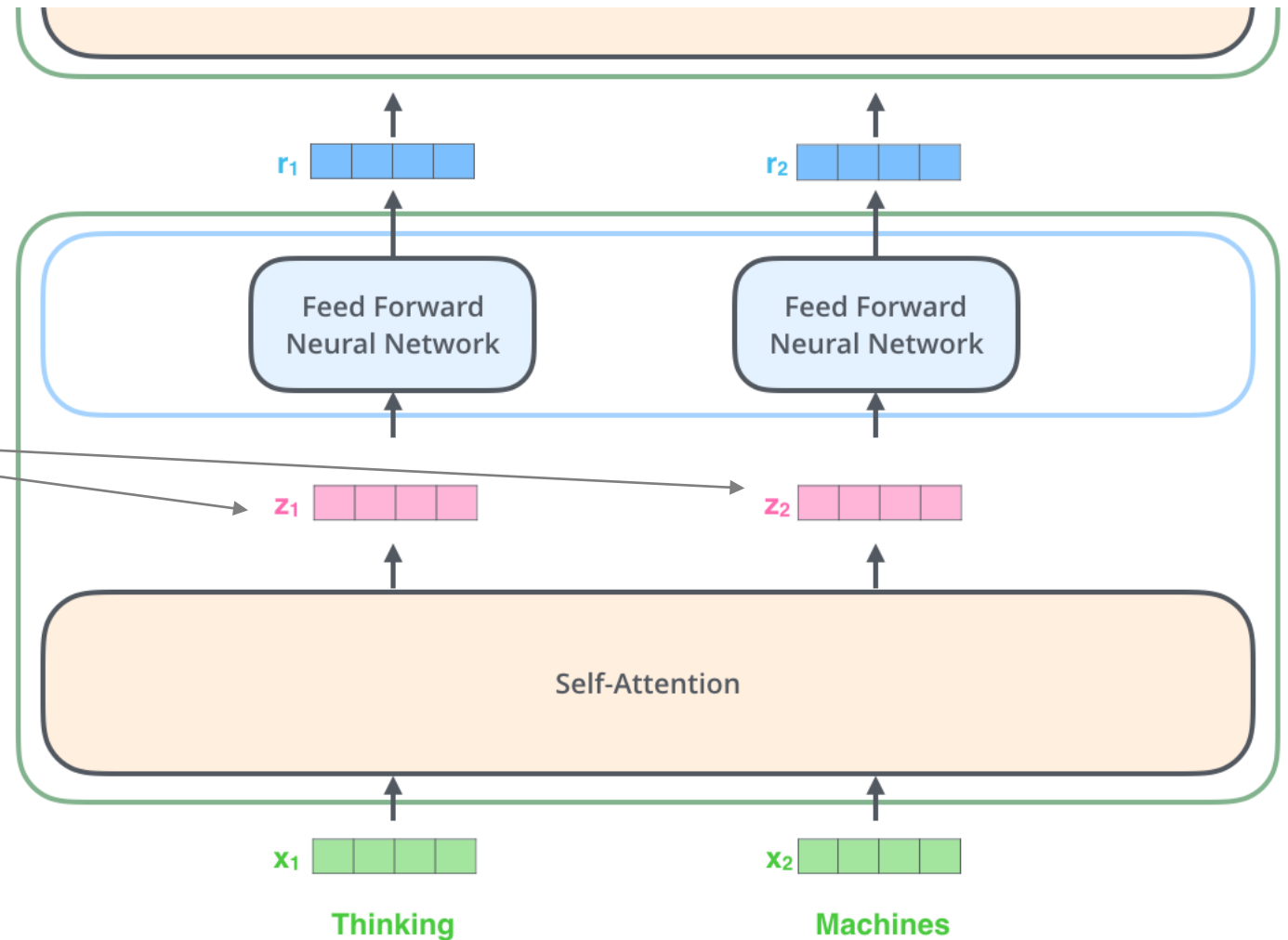
Encoder Block Map

- Encoder block breaks down into two main parts: Self-Attention, and Feed Forward layers.



Encoder Block Map

- Encoder block breaks down into two main parts: Self-Attention, and Feed Forward layers.
- Self-Attention layer is applied to each word individually.

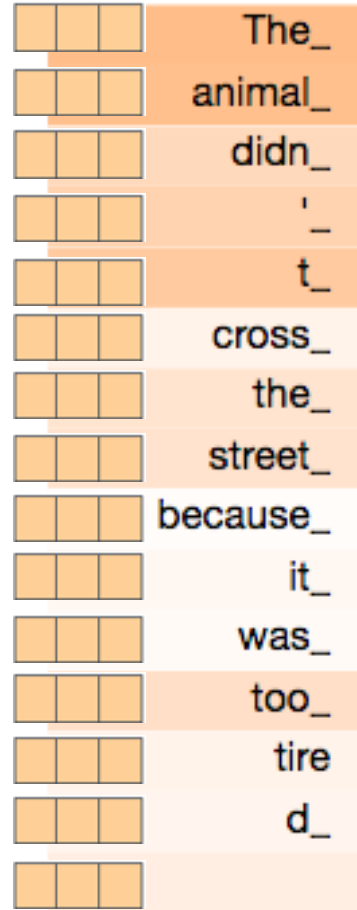


Self-Attention: Overview

How it works:

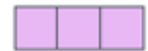
1. To determine how much attention a word should pay to each other other, we compute a **query vector** for the word and compare it to a **key vector** for every other word...

Key vectors



The_
animal_
didn_
'_
t_
cross_
the_
street_
because_
it_
was_
too_
tire
d_

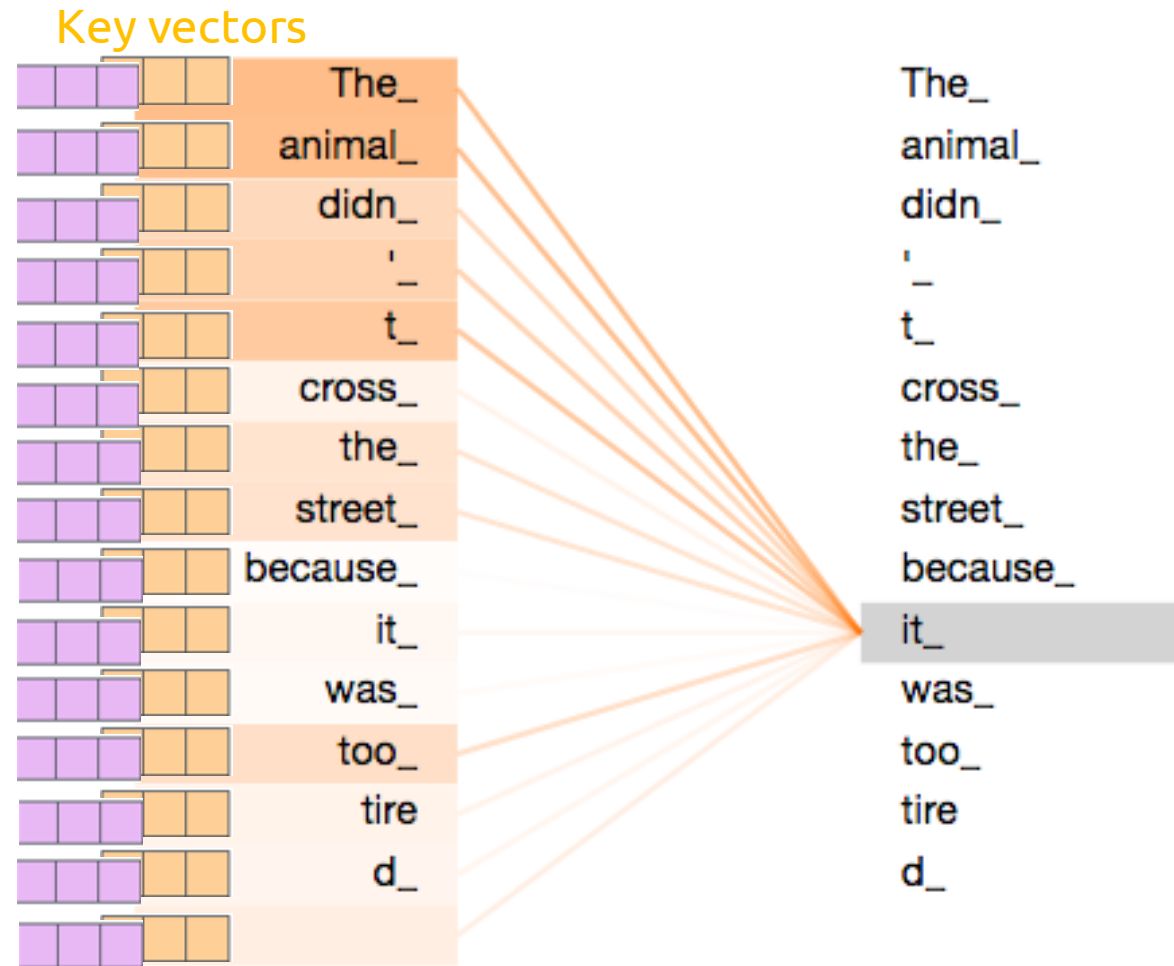
Query vector



Self-Attention: Overview

How it works:

1. To determine how much attention a word should pay to each other other, we compute a **query vector** for the word and compare it to a **key vector** for every other word...

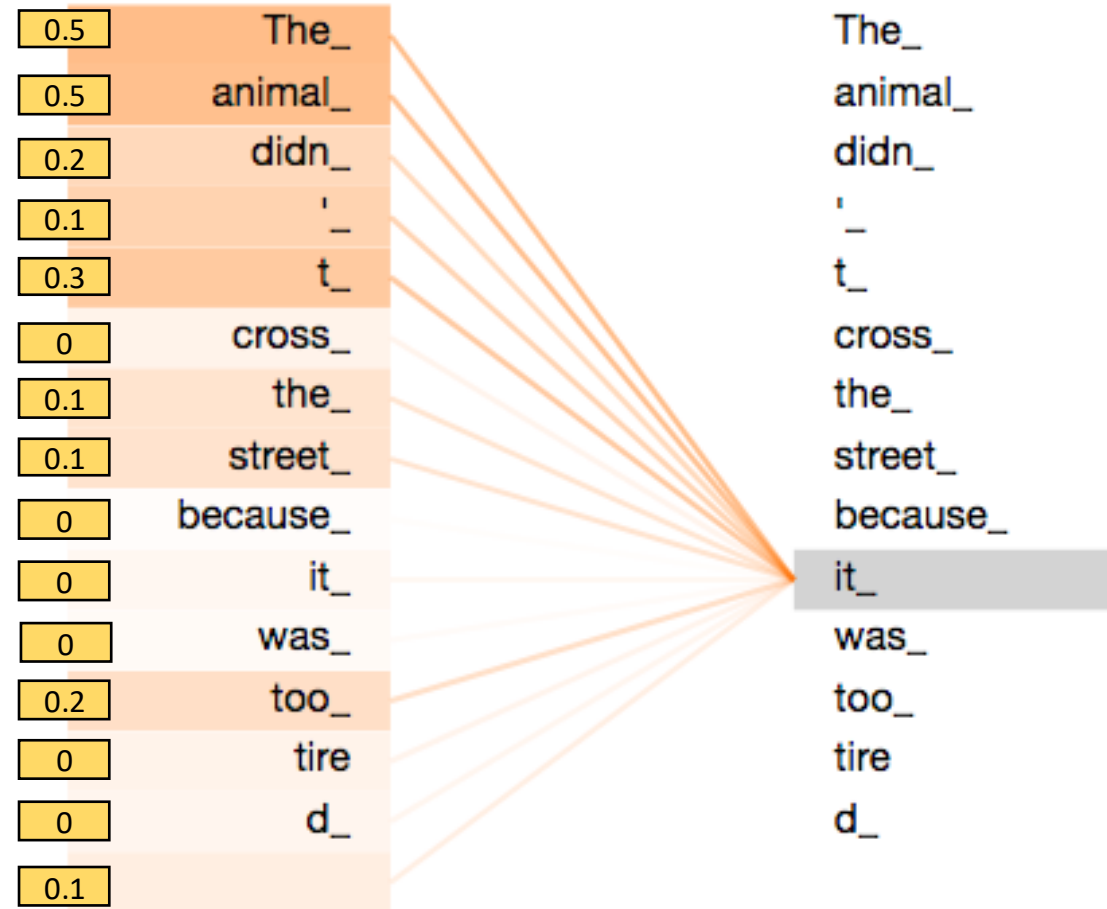


Self-Attention: Overview

How it works:

1. To determine how much attention a word should pay to each other other, we compute a **query vector** for the word and compare it to a **key vector** for every other word... **to compute our alignment score**

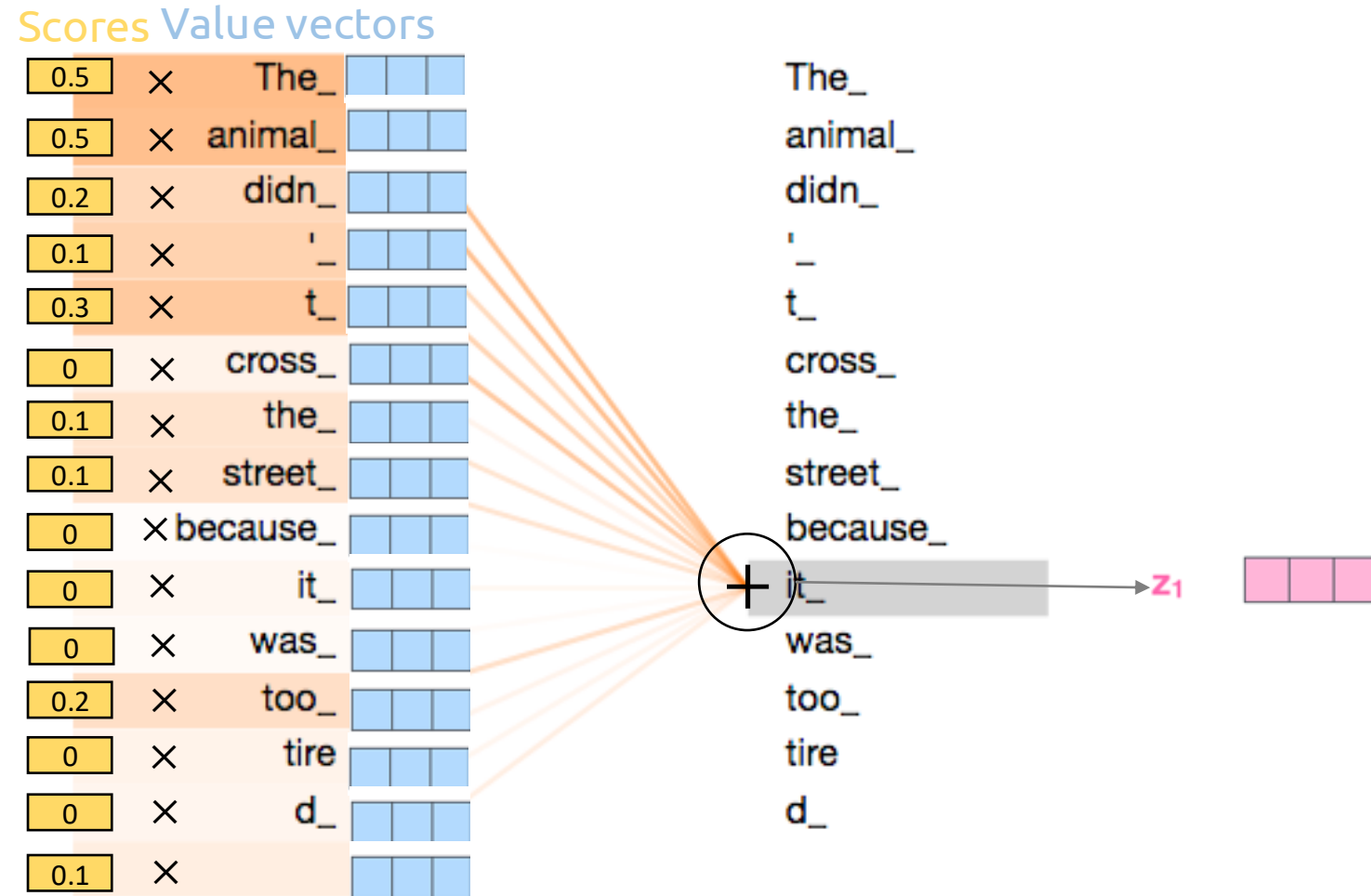
Scores



Self-Attention: Overview

How it works:

1. To determine how much attention a word should pay to each other other, we compute a **query vector** for the word and compare it to a **key vector** for every other word... **to compute our alignment score**
2. To produce the output vector, we sum up the **value vectors** for each word, weighted by the score we computed in step 1



Self-Attention: Details

Computing self-attention for “Thinking”

1. **Score:** Dot product query vector for “Thinking” (q_1) with the key vectors of each word in the sentence ($k_{1,2,...,n}$).
2. **Scale:** Divide each score by square root of key vector dimensionality. Results in more stable gradients.
3. **Softmax:** Apply softmax layer.
4. **Weighting:** Multiply value vector of each word in the sentence ($v_{1,2,...,n}$) with the respective softmax values.
5. **Sum:** Sum up weighted value vectors ($v_{1,2,...,n}$) into one final self-attention vector for “Thinking” (z_1)

We are weighting the *context* provided by each word by the amount of *attention* we should pay.

Input

Embedding

Queries

Keys

Values

Score

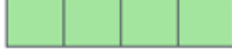
Divide by 8 ($\sqrt{d_k}$)

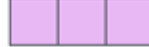
Softmax

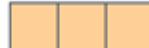
Softmax
X
Value

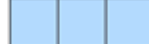
Sum

Thinking

x_1 

q_1 

k_1 

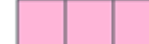
v_1 

$q_1 \cdot k_1 = 112$

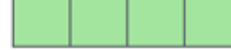
14

0.88

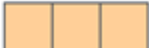
v_1 

z_1 

Machines

x_2 

q_2 

k_2 

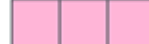
v_2 

$q_1 \cdot k_2 = 96$

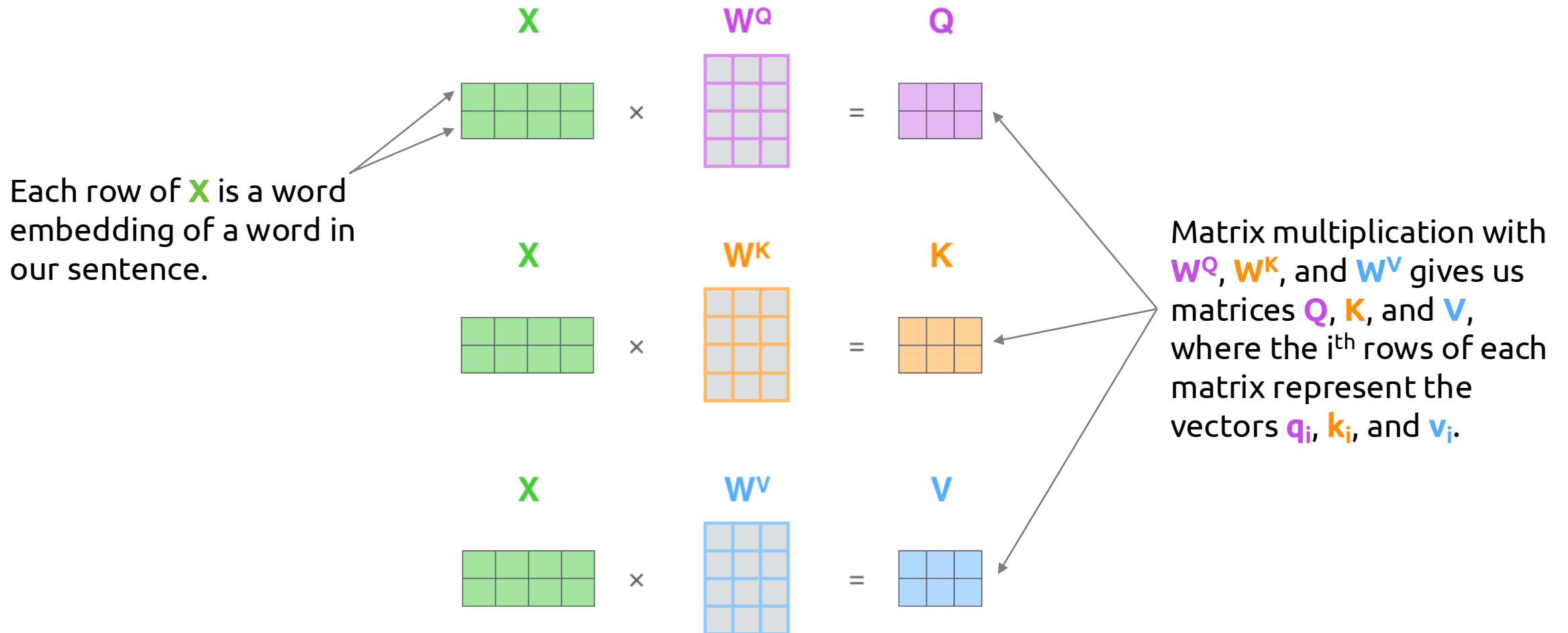
12

0.12

v_2 

z_2 

Self-Attention as a Matrix Computation



Self-Attention as a Matrix Computation

Matrix multiplying **Q** and the transpose of **K** calculates all the "score" values.

$$\text{softmax} \left(\frac{\begin{matrix} \text{Q} \\ \text{3x3 grid} \end{matrix} \times \begin{matrix} \text{K}^T \\ \text{3x3 grid} \end{matrix}}{\sqrt{d_k}} \right)$$

Dividing by $\sqrt{d_k}$ correctly scales values.

The result is a **Z** matrix where the i^{th} row represents the self-attention vector **z_i**

$$\begin{matrix} \text{V} \\ \text{3x3 grid} \end{matrix} = \begin{matrix} \text{Z} \\ \text{3x3 grid} \end{matrix}$$

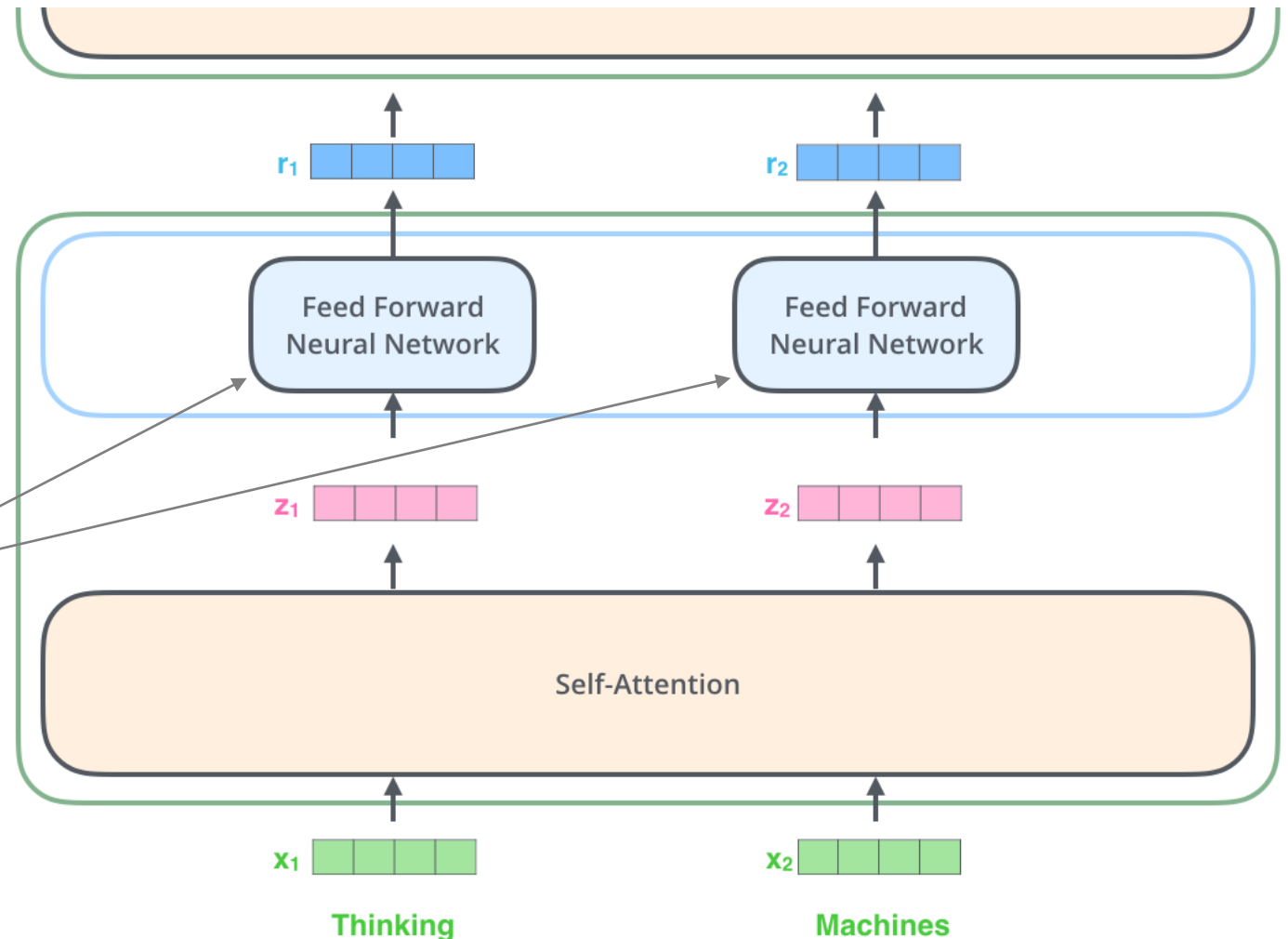
Multiplying the resulting vector with **V** properly weighs the **v_i** vectors.

Any questions?



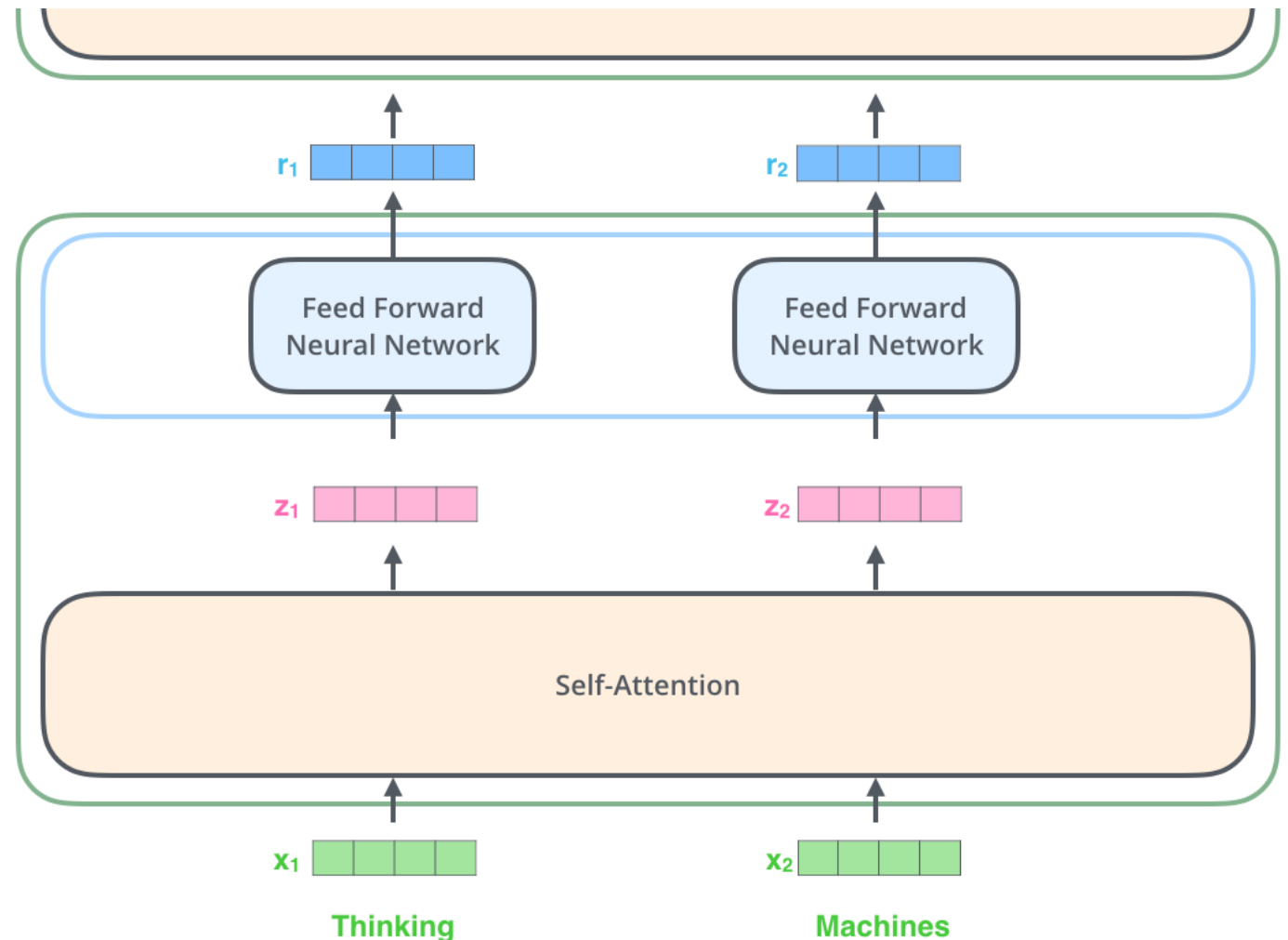
Encoder Block Map

- Encoder block breaks down into two main parts: Self-Attention, and Feed Forward layers.
- Self-Attention layer is applied to each word individually.
- Feed Forward layer is applied to each word individually.
- The outputs of the feed forward layer are then passed as the inputs of the next encoder block.



Encoder Block Map

- Encoder block breaks down into two main parts: Self-Attention, and Feed Forward layers.
- Self-Attention layer is applied to each word individually.
- Feed Forward layer is applied to each word individually.
- The outputs of the feed forward layer are then passed as the inputs of the next encoder block.
- *But we forgot about something...*



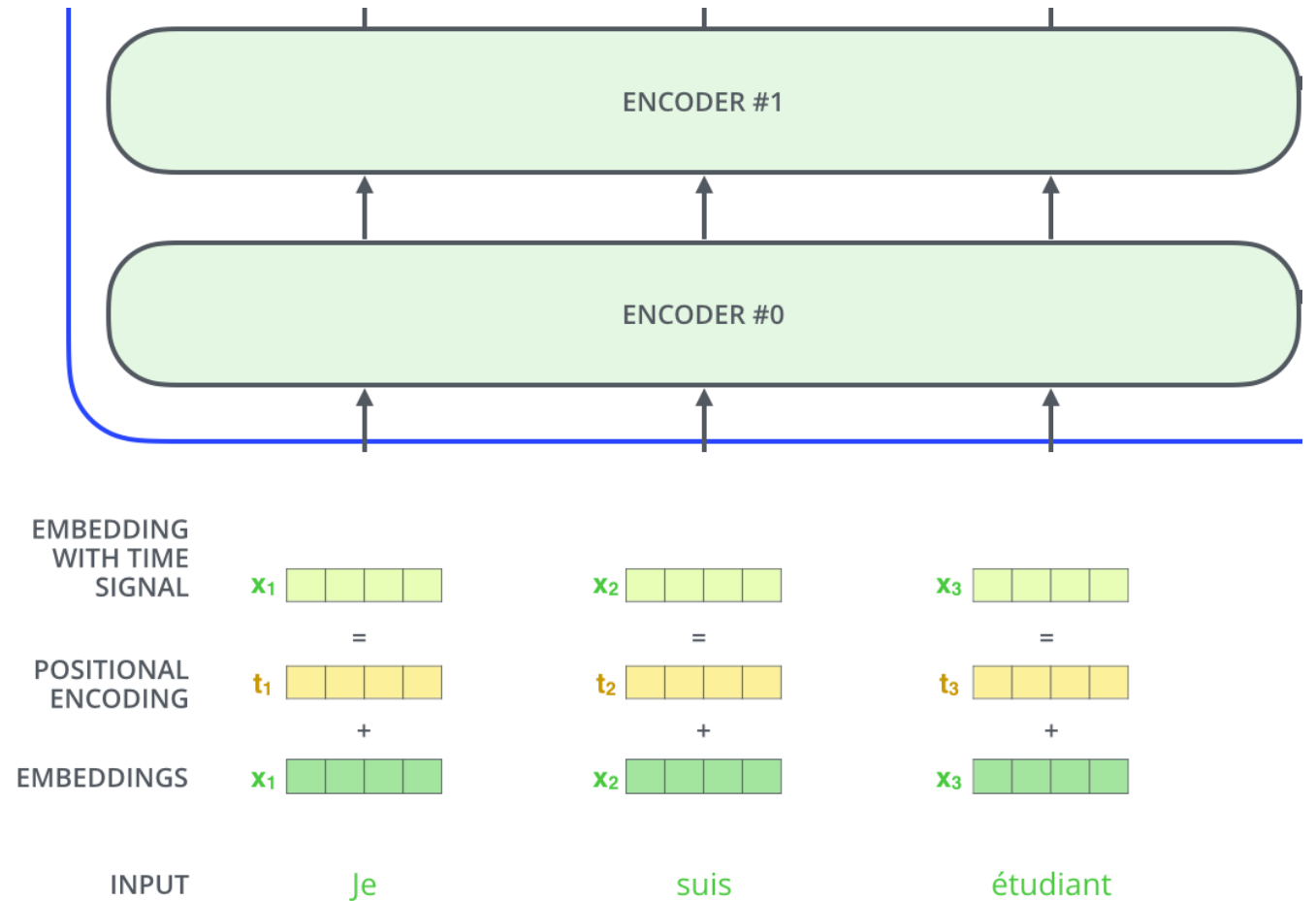
What are we missing?

Hint: Remember – we are not using RNNs anymore.

Have we neglected/lost any information about the original input sequence?

Positional Encodings

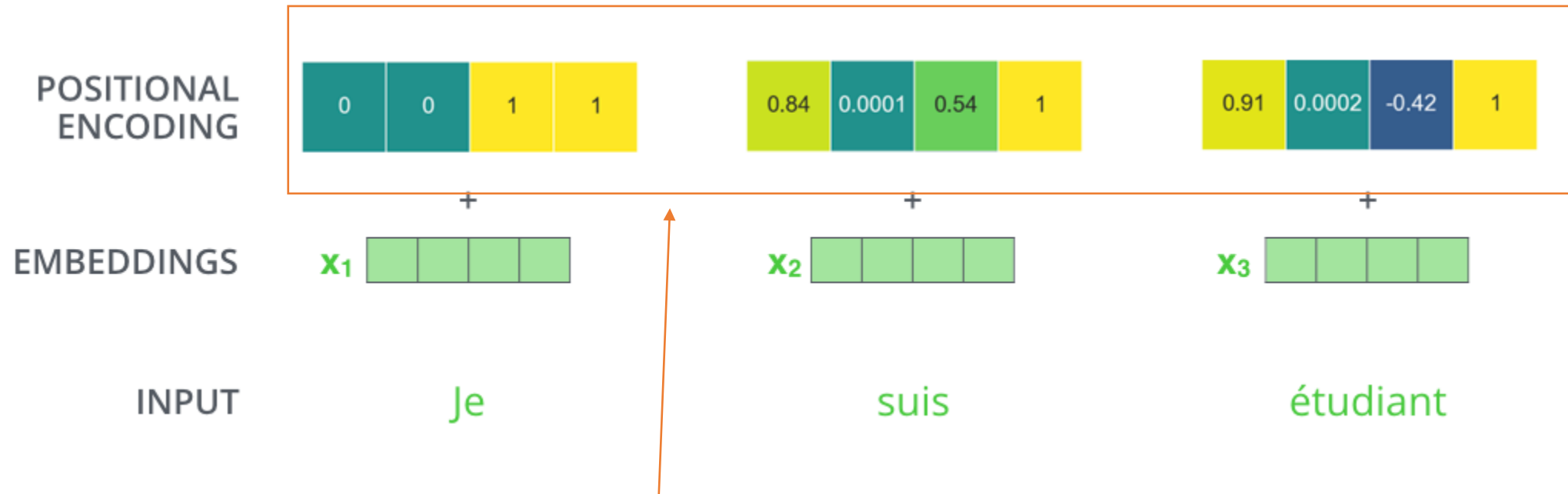
- Instead of passing Embedding vector to encoder, we pass **Embedding with Time Signal** vector.
- Positional Encoding is `embedding_size` vector that encodes information about the position of a word in a sequence.
- Positional Encodings can be learned or defined by a fixed function.
- We add the Positional Encoding to the Embedding to get our Embedding with Time Signal vector.



Positional Encodings



Positional Encodings

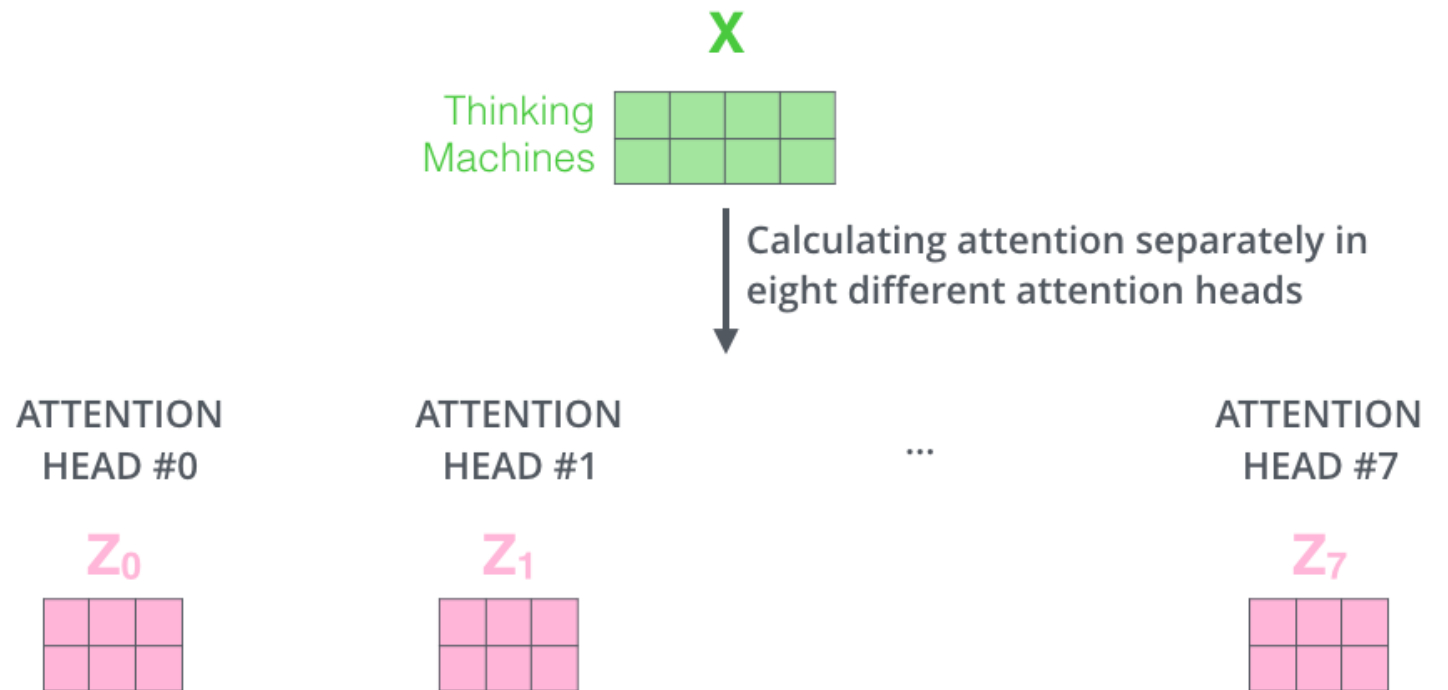


Where do these numbers come from?

- Carefully-chosen sinusoidal patterns such that when we add them to the embedding vectors, their dot products w/ each other reflect the distance between them in the sentence.

Multi-head Attention

- Multi-head Attention is used to improve the performance of regular self-attention.
- We compute self-attention as before some number of times. Call these “attention heads”.
- The size of the attention heads are smaller than when just using regular self-attention.



Multi-head Attention



Multi-head Attention

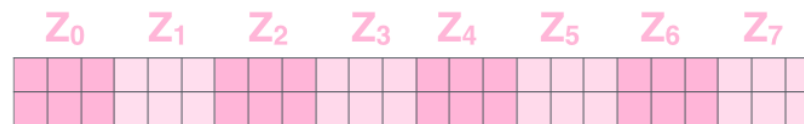
1) Concatenate all the attention heads



To get one set of attention vectors, we concatenate all the heads and apply a linear layer in order to get Z .

Multi-head Attention

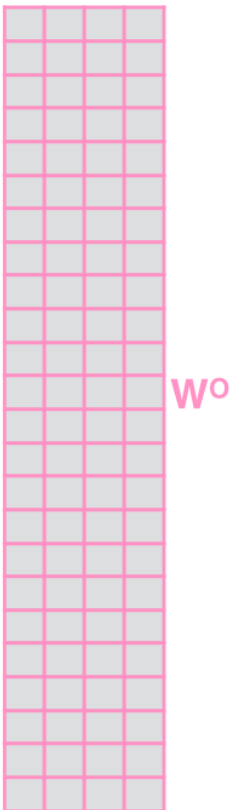
1) Concatenate all the attention heads



To get one set of attention vectors, we concatenate all the heads and apply a linear layer in order to get Z .

2) Multiply with a weight matrix W^O that was trained jointly with the model

$\times$



Multi-head Attention

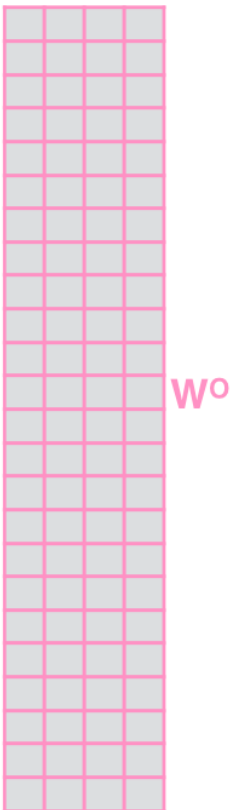
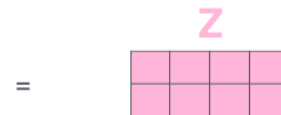
1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

$\times$

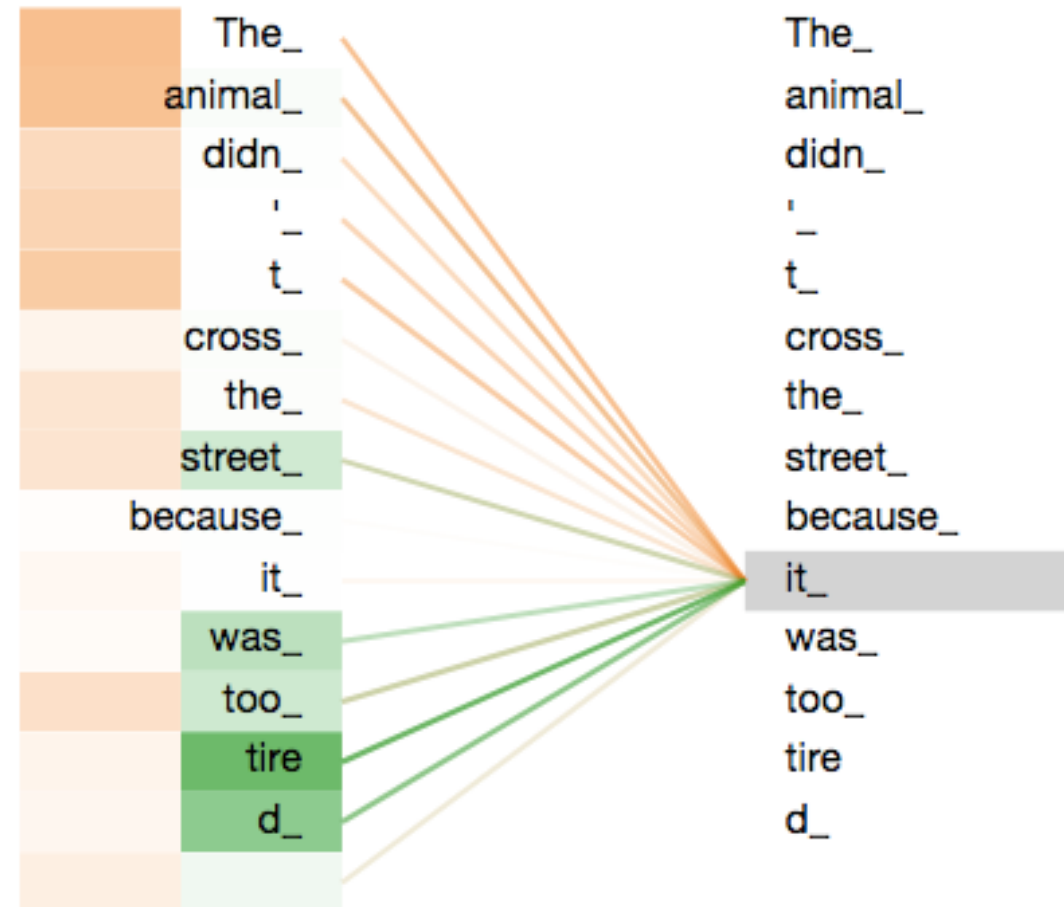
3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



To get one set of attention vectors, we concatenate all the heads and apply a linear layer in order to get Z .

Multi-head Attention Visualized

Multiple heads allow for each head to learn different relationships between words in the sentence.



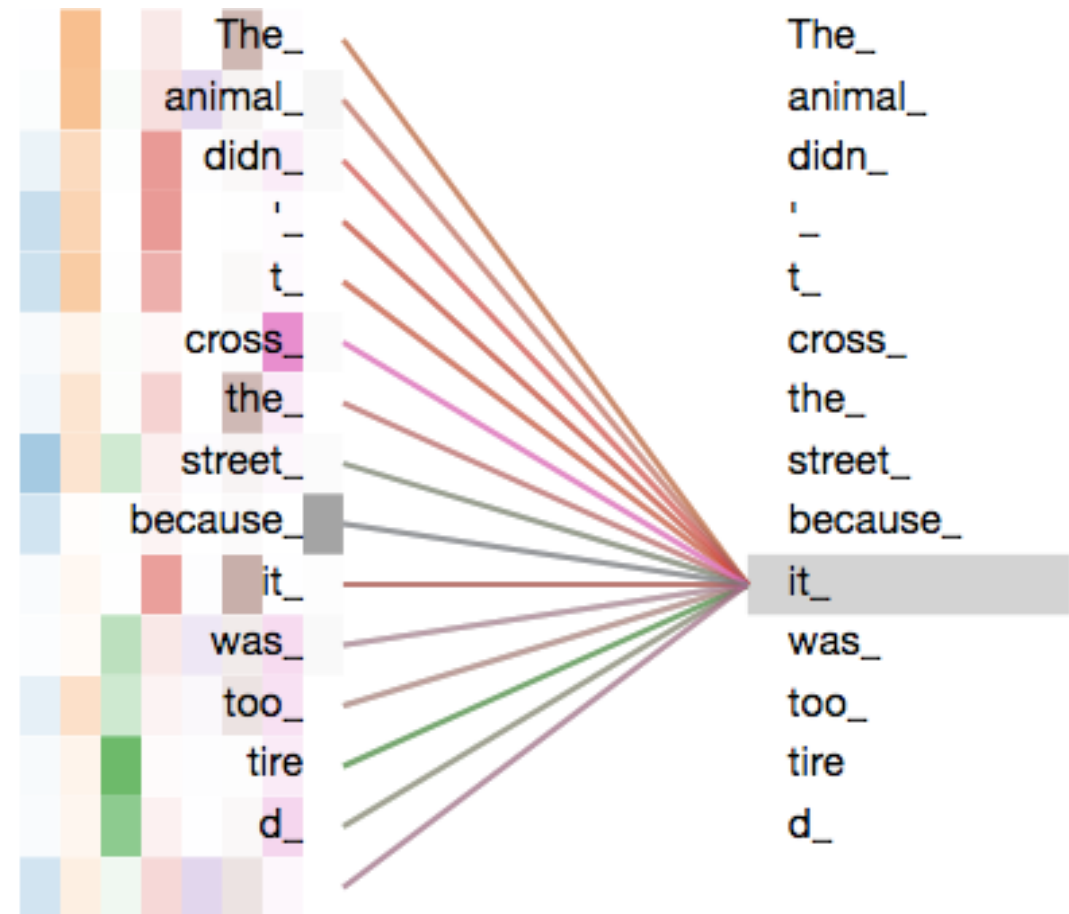
Multi-head Attention Visualized

Multiple heads allow for each head to learn different relationships between words in the sentence.

These relationships become more difficult to interpret with many heads.

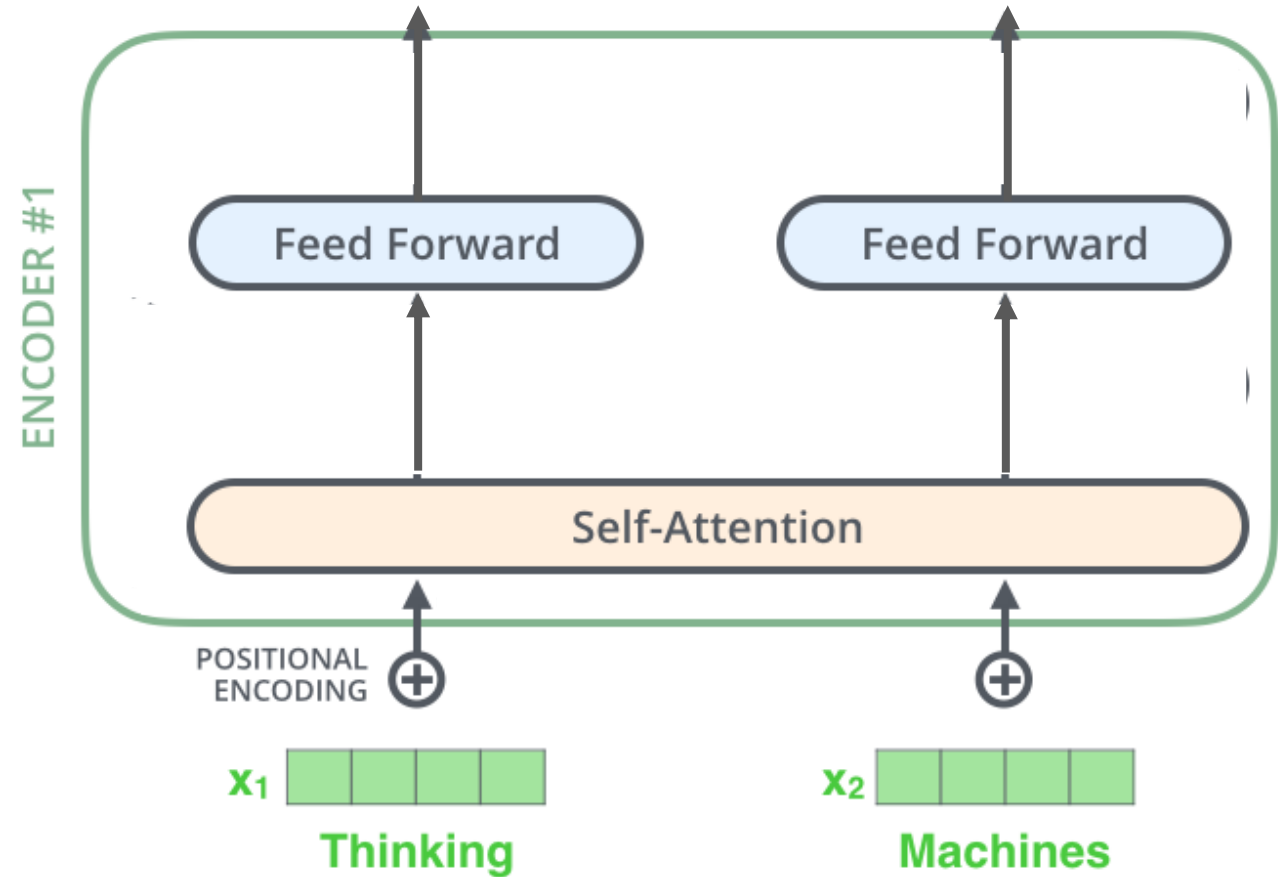
Visualizer tool:

https://colab.research.google.com/github/tensorflow/tensor2tensor/blob/master/tensor2tensor/notebooks/hello_t2t.ipynb



Extra Performance Improvements

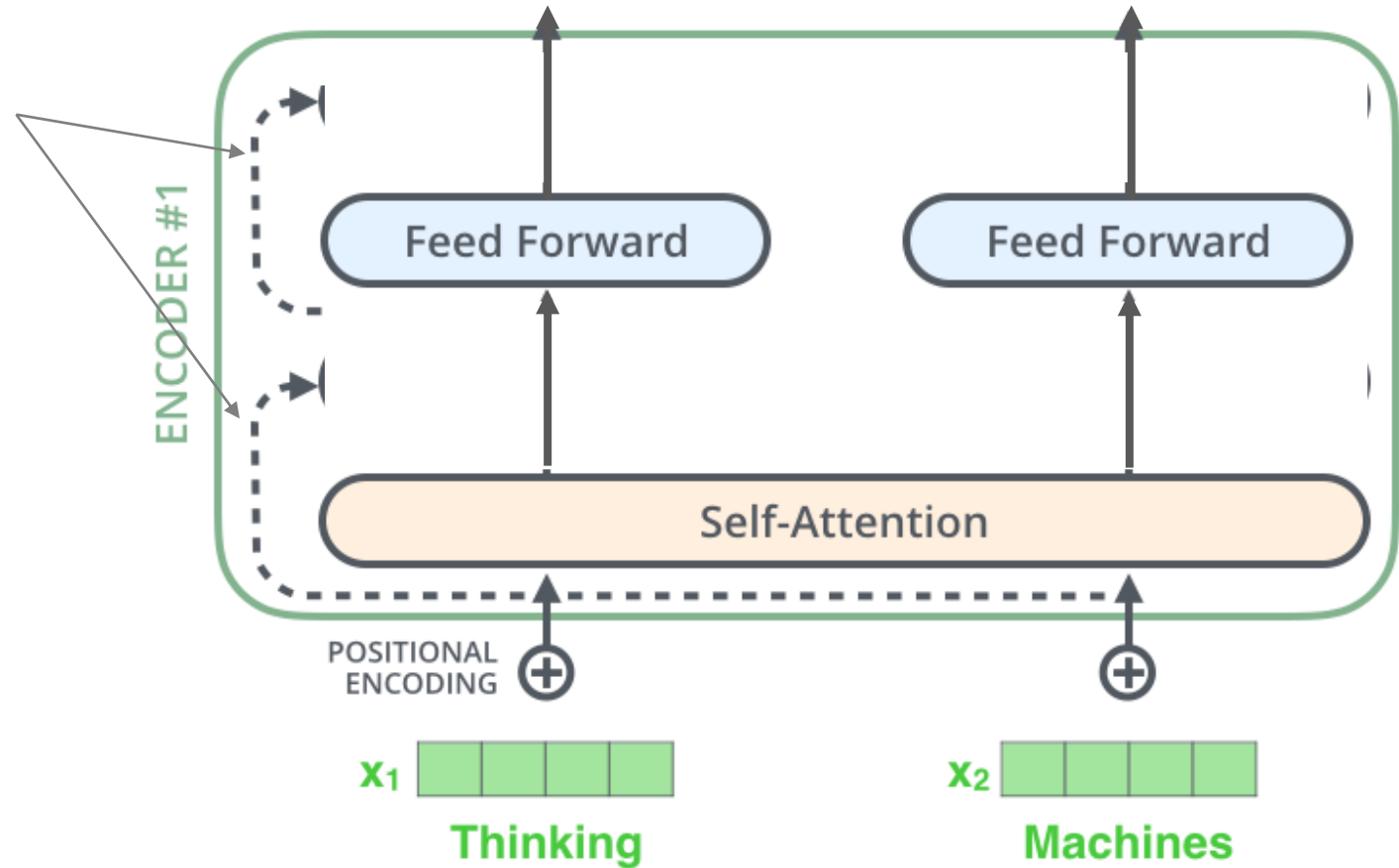
- **Recap:** This is the current state of our encoder block



Extra Performance Improvements

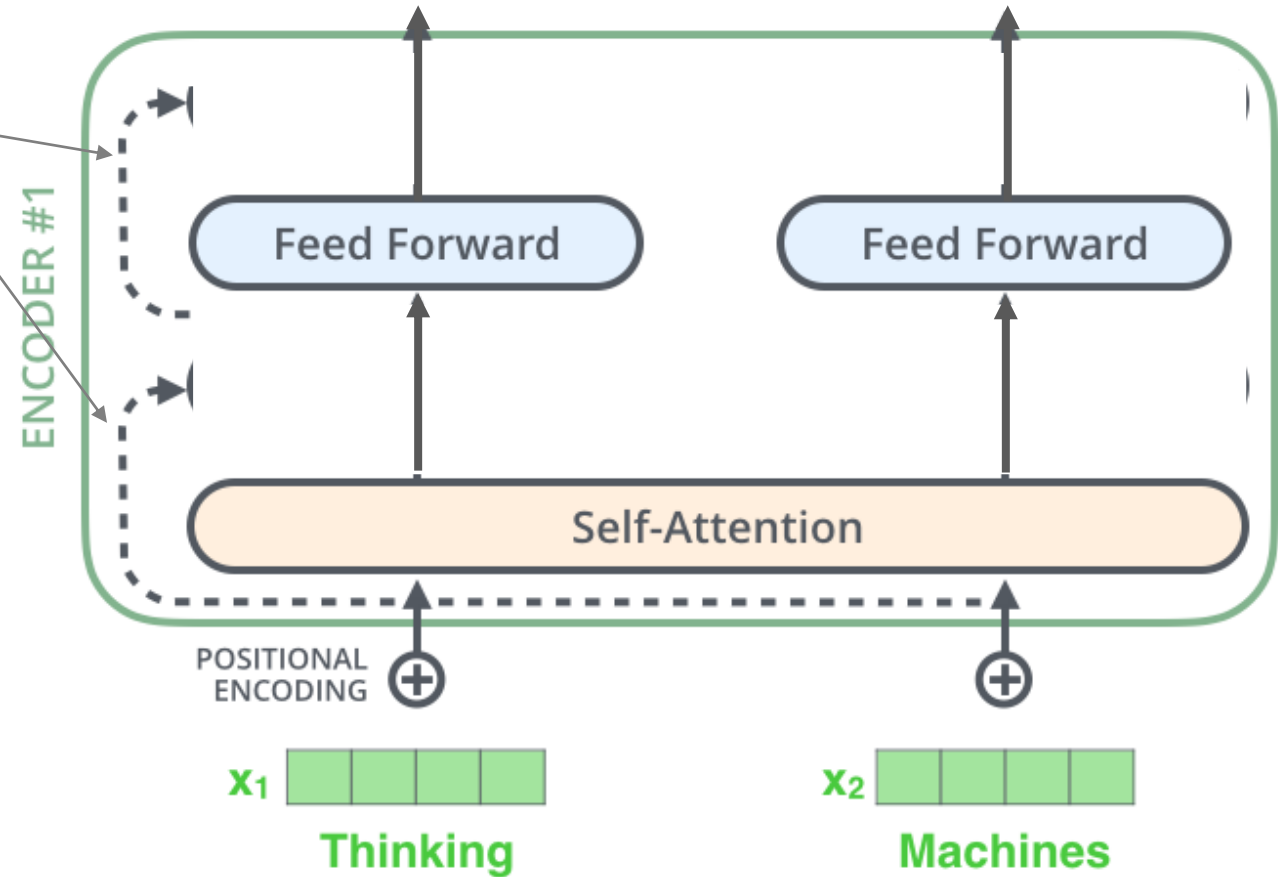
- **Residual Connections**

Do you remember when we talked about these?



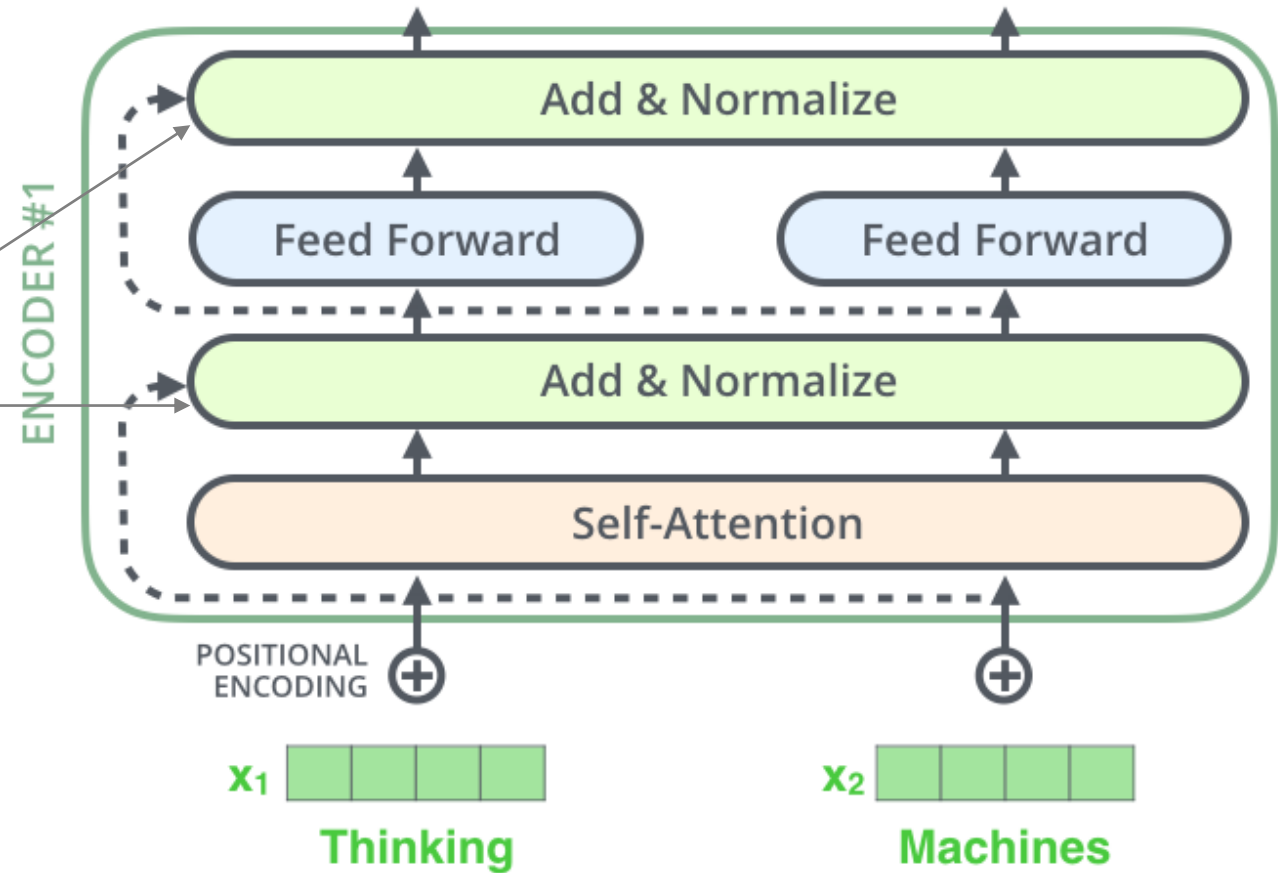
Extra Performance Improvements

- **Residual Connections:** Just like in CNN architectures, transformer models make use of skip connections to negate vanishing gradients.



Extra Performance Improvements

- **Residual Connections:** Just like in CNN architectures, transformer models make use of skip connections to negate vanishing gradients.
- **LayerNorm:** Similar to Batch Normalization, improves convergence time.

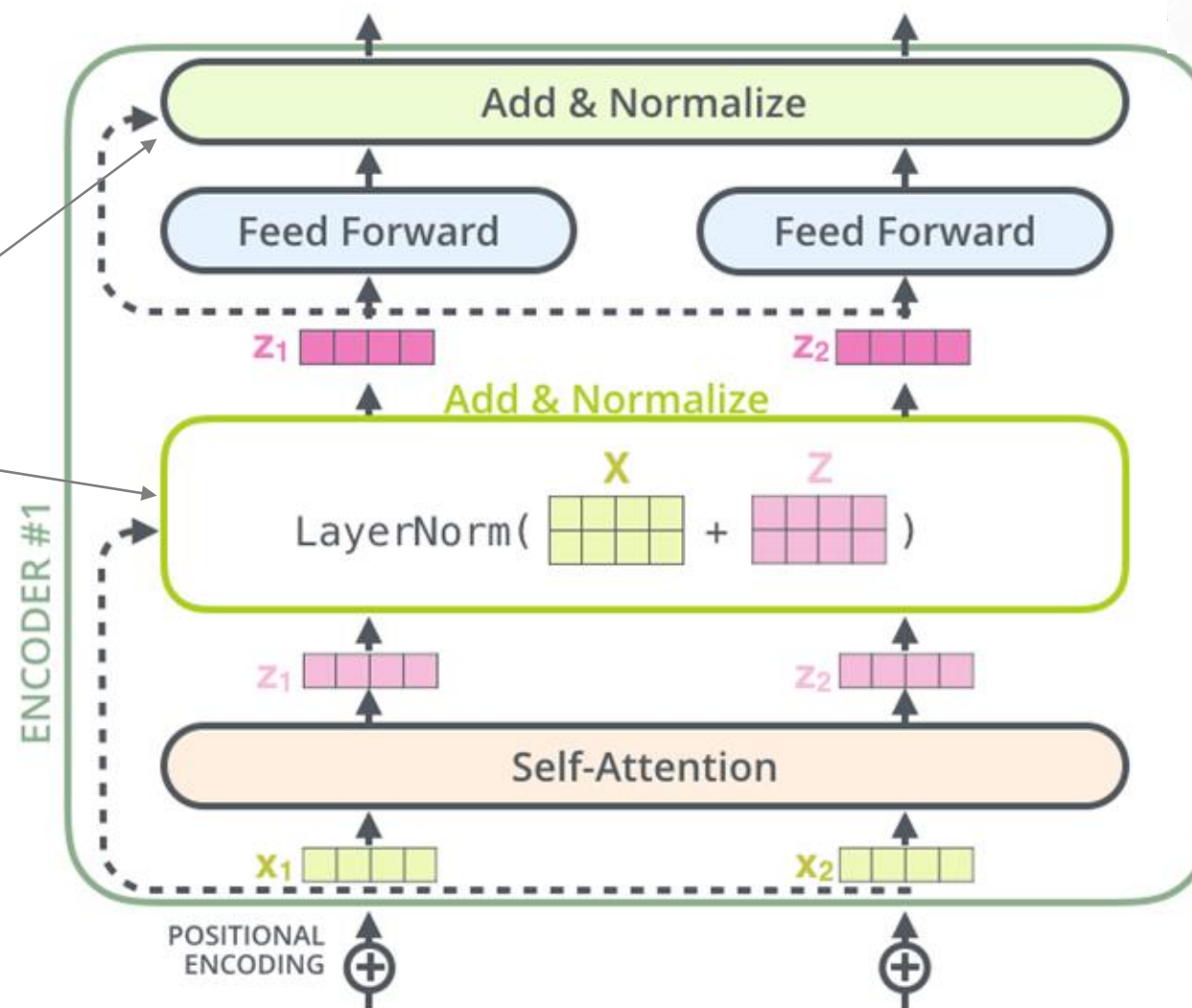




Extra Performance Improvements

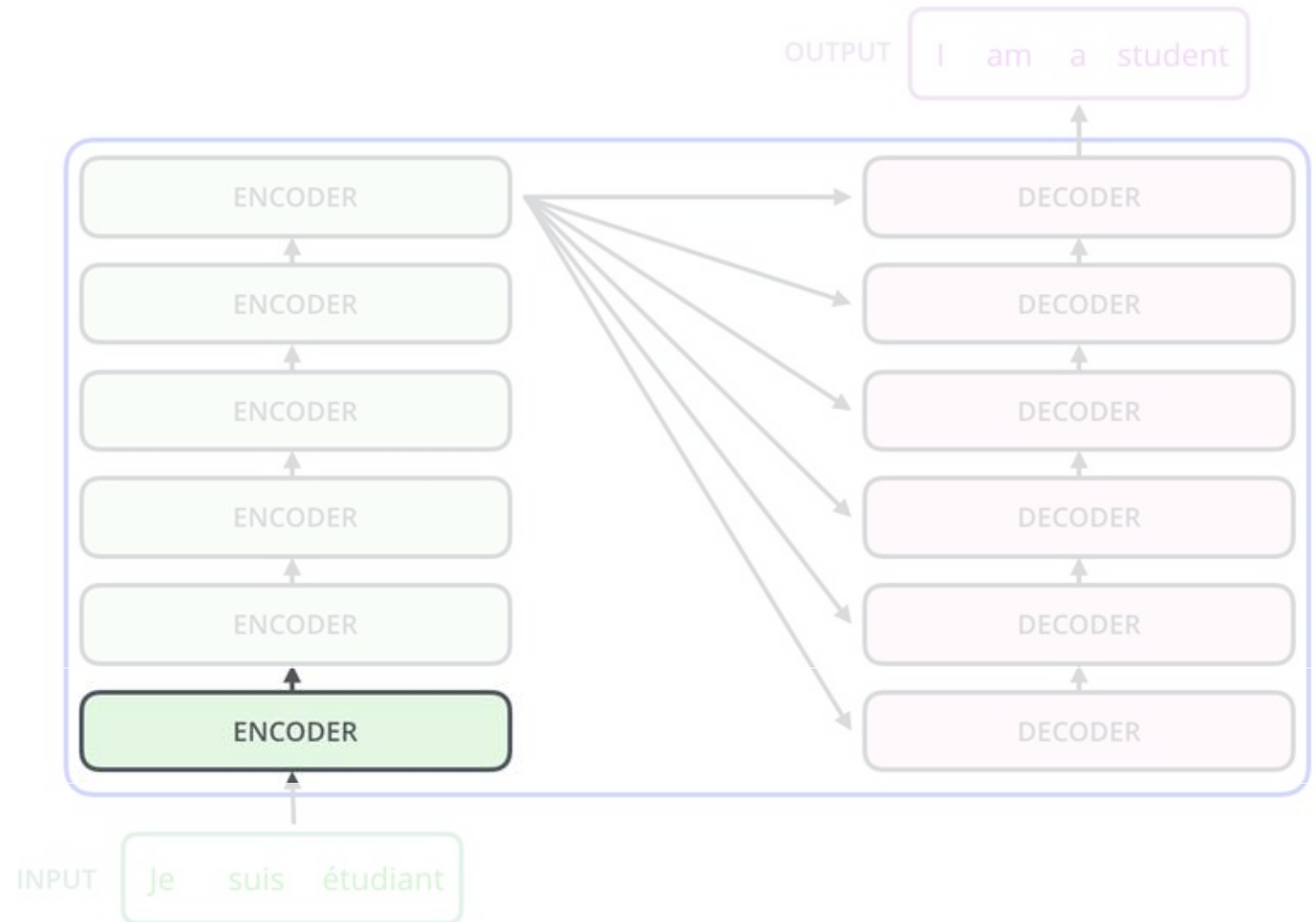
- **Residual Connections:** Just like in CNN architectures, transformer models make use of skip connections to negate vanishing gradients.
- **LayerNorm:** Similar to Batch Normalization, improves convergence time.

We combine the residual connection and self-attention layer by adding them together.

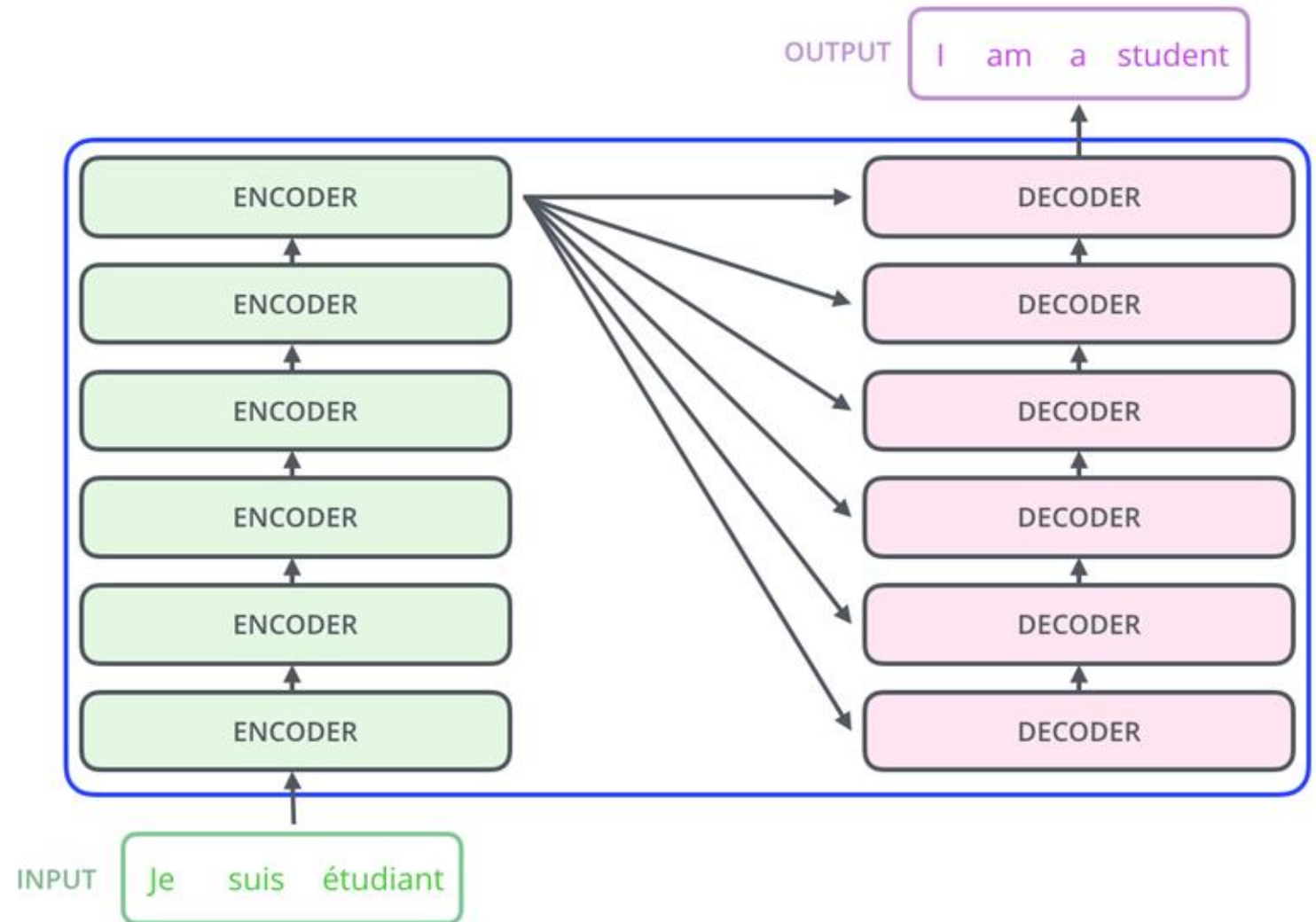


Decoder

Transformer Model Overview

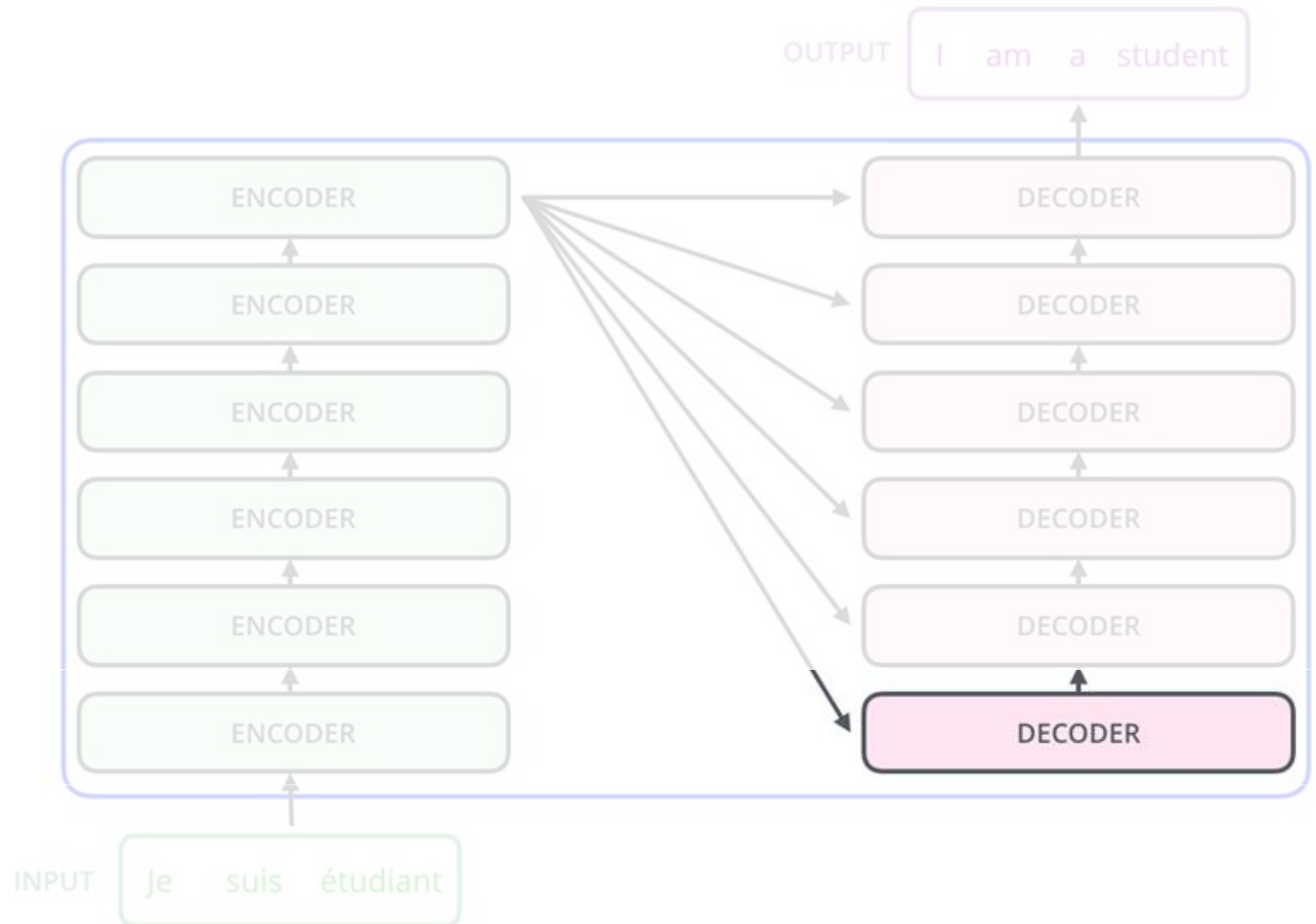


Transformer Model Overview



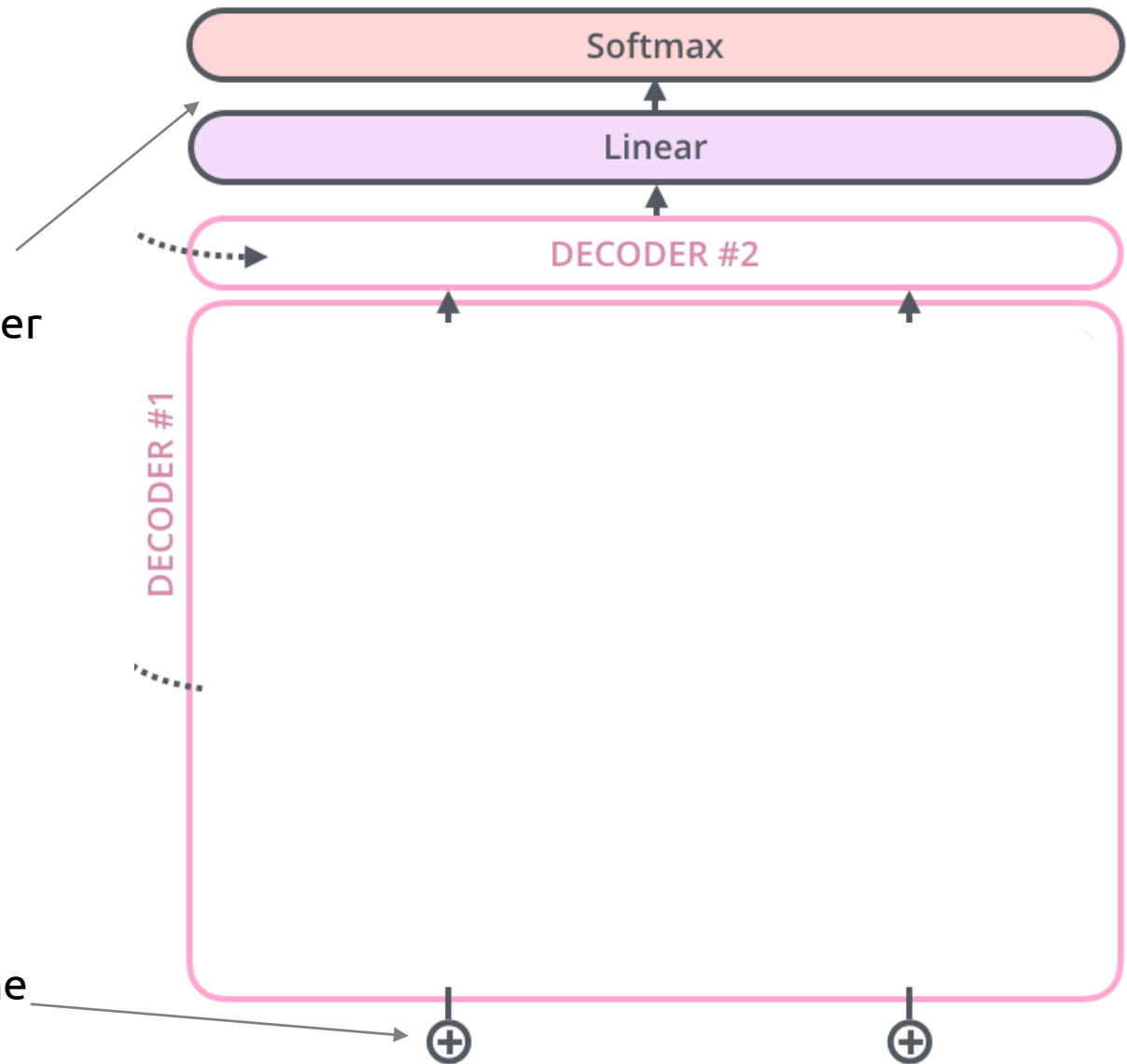
Transformer Model Overview

- Now let's look under the hood of a Decoder block



Decoder Block

Ultimately, decoder terminates in a linear + softmax to predict a probability distribution over the next word in the target language (same as with seq2seq model)

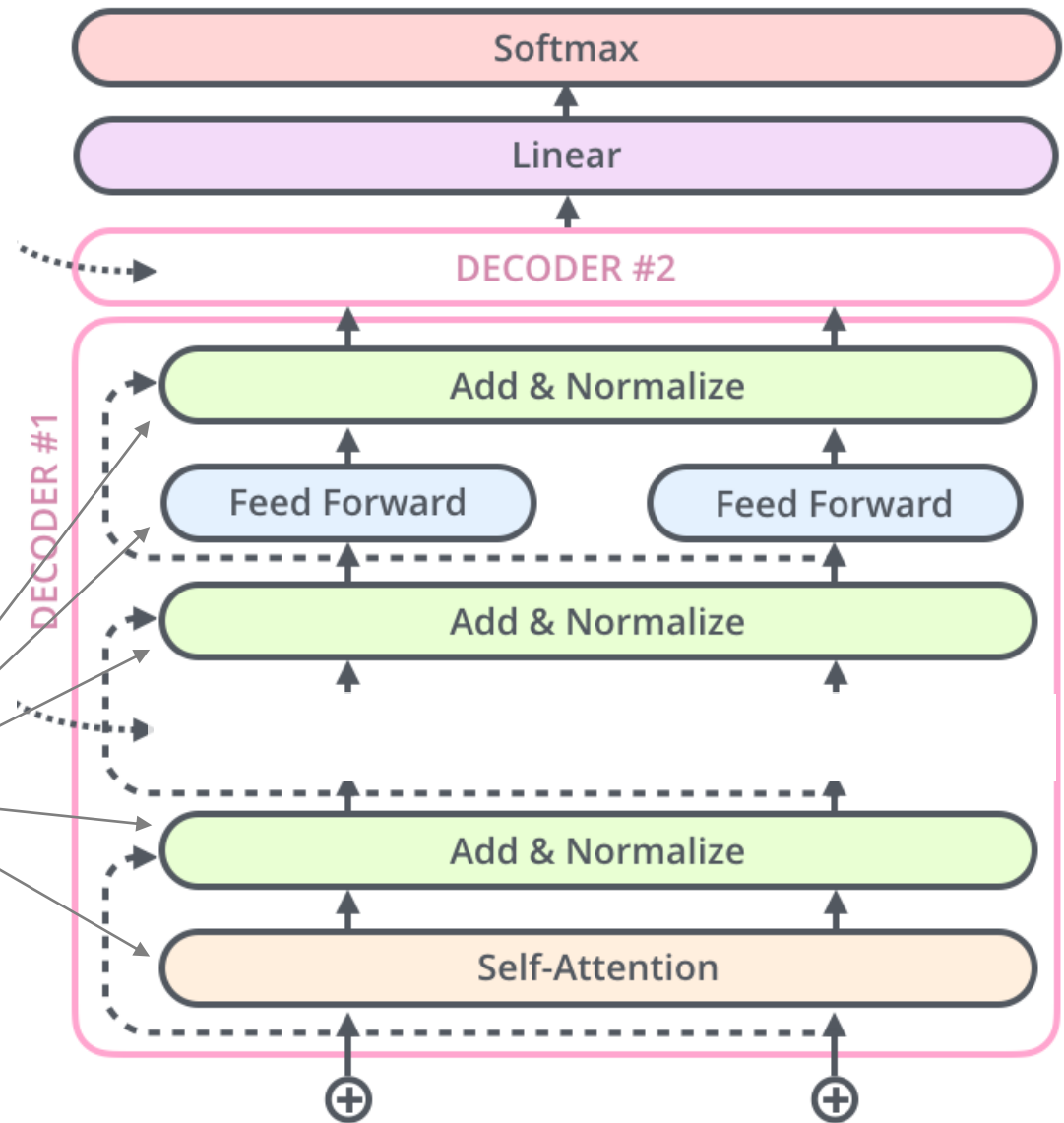


Like in other seq2seq models, the decoder receive the target sequence shifted back by one step as input.

Decoder Block

What else do we need here?

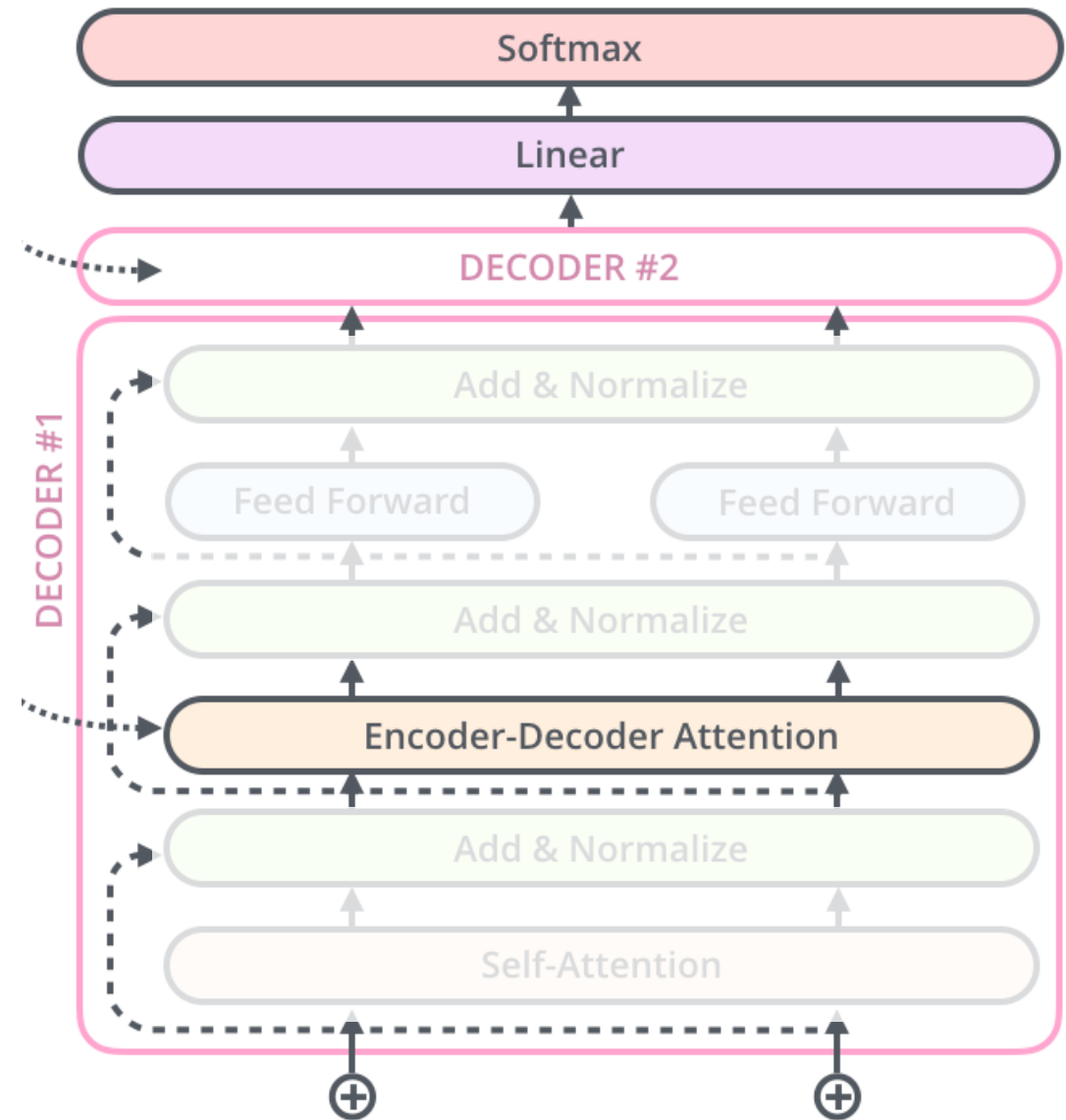
Self-attention, LayerNorm, and Feed Forward layers are identical to the Encoder block versions.



Decoder Block

What's Encoder-Decoder Attention?

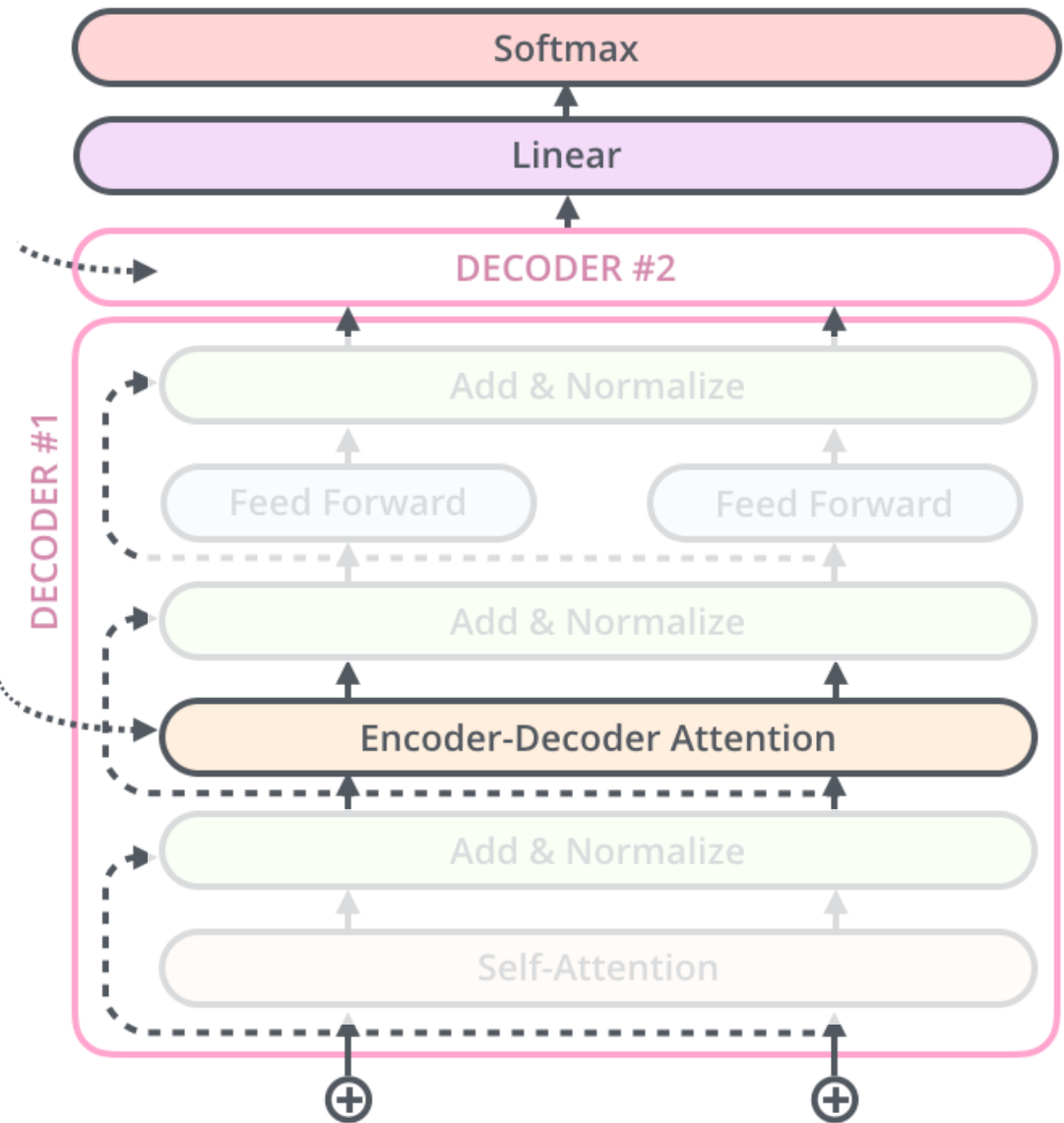
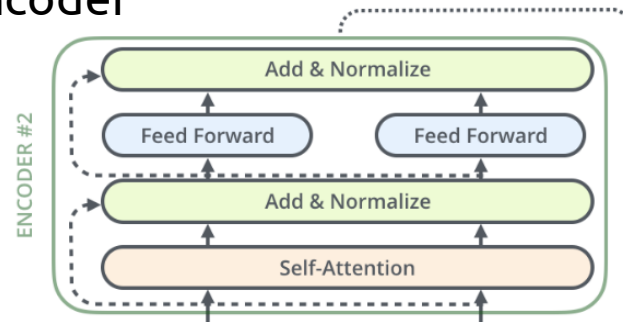
- The part that says “how much should each *output* word pay attention to each input word”
- Analogous to the ‘weighted average of LSTM states’ that we pass to each decoder step in the seq2seq model



Decoder Block

How to implement Encoder-Decoder attention?

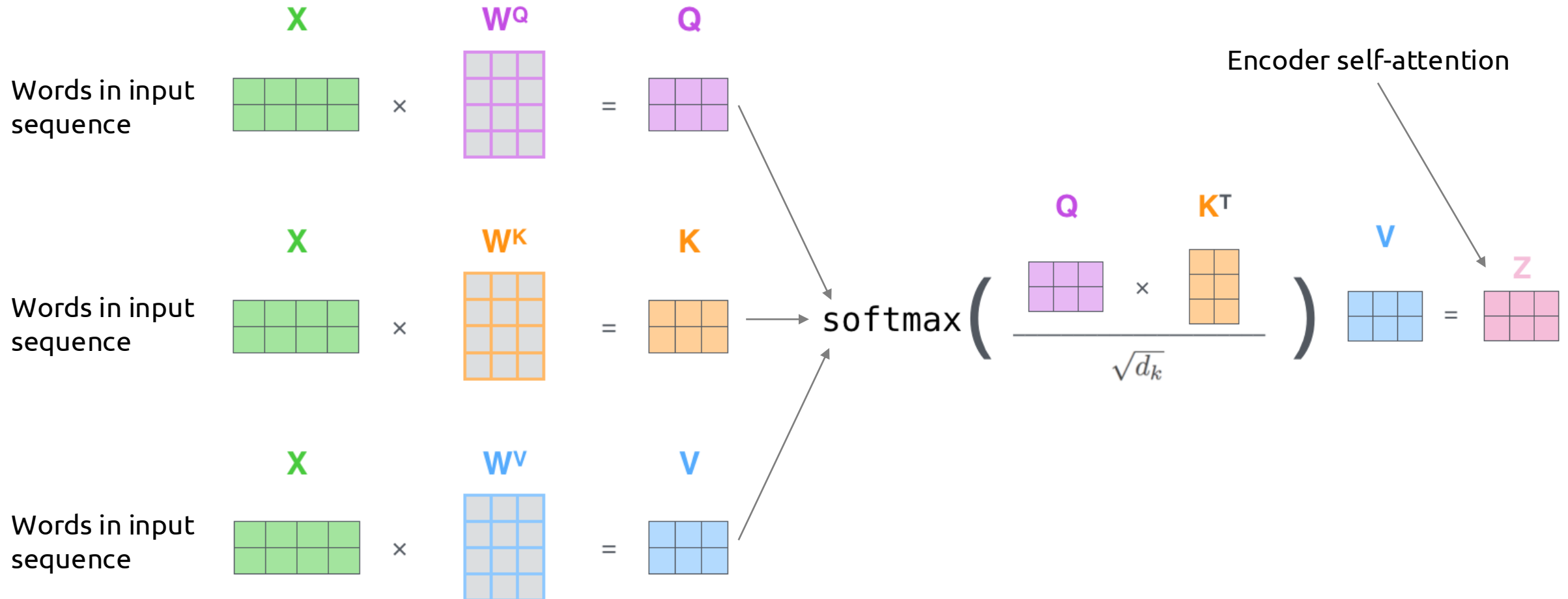
- It's exactly the same algorithm as self-attention...
- ...except that it queries the source sentence, instead of the target sentence
- Specifically, it extracts the **K** and **V** vectors from the output of the encoder



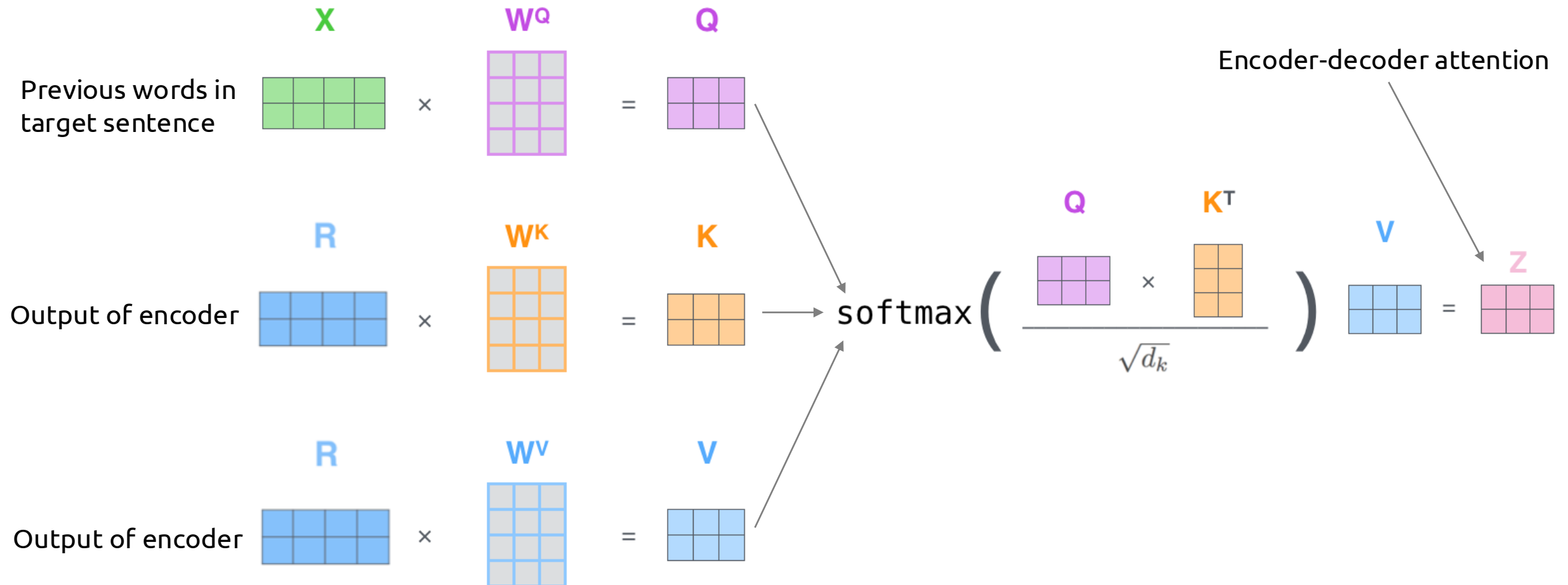
Encoder Self-Attention

What do we change for Encoder-Decoder attention?

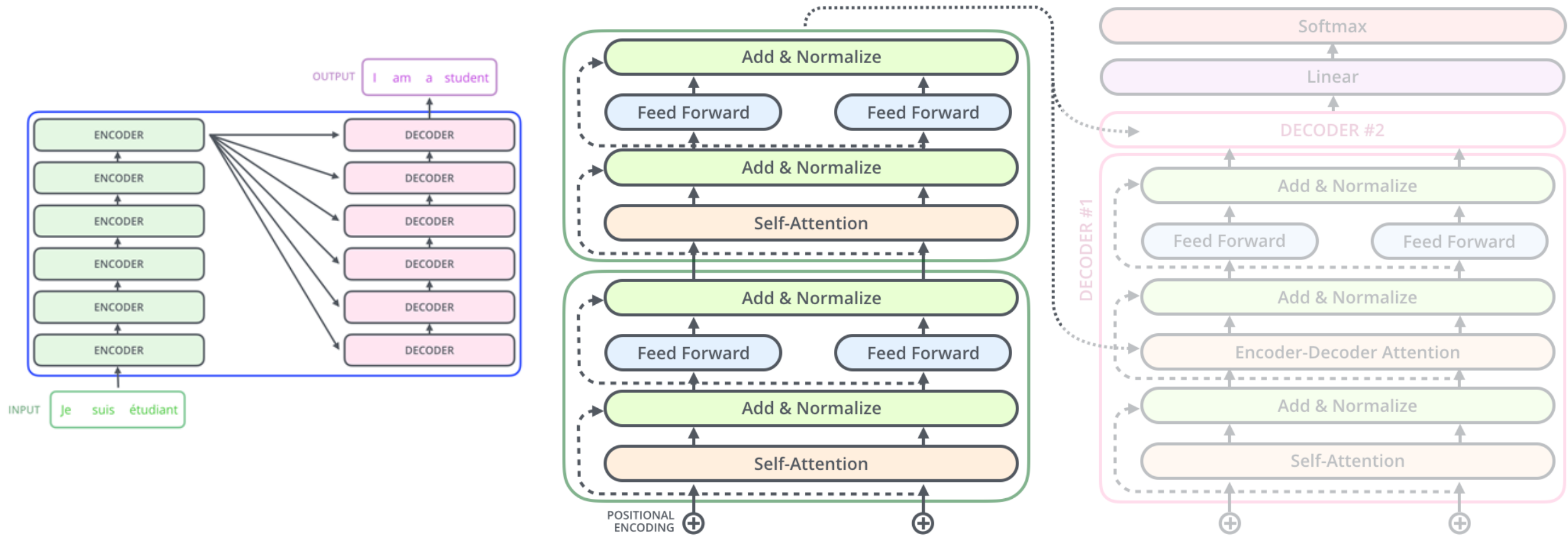
What will be our query?
What will be our keys and values?



Encoder-Decoder Attention

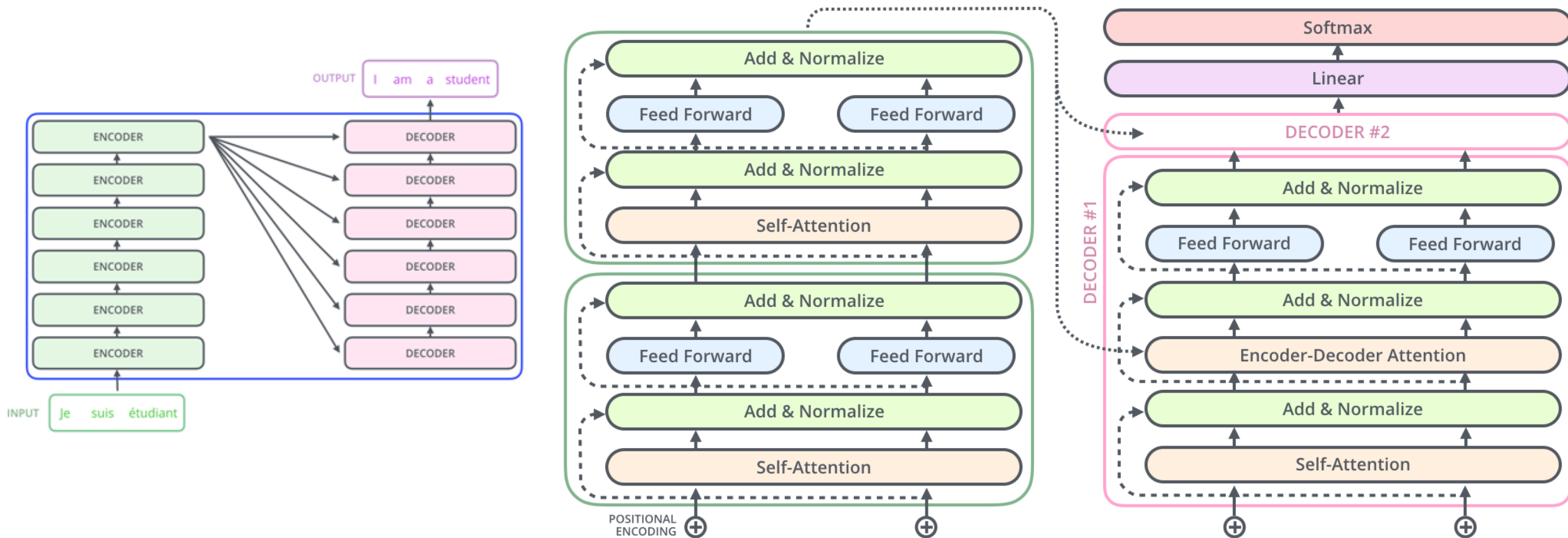


Encoders and Decoders Together





Encoders and Decoders Together



Side-note: Masking

Implementing the decoder side of self-attention uses *masking*.

Masking is a technique used to nullify certain words before they are passed to the model to prevent the model from seeing them.

The reason for this stems from the fact that for the decoder, we would like to pass the entire sequence of *previous* words.

In practice, it is a lot easier to instead pass it the entire sequence, and mask out all of the words that the model isn't allowed to see.

Code: Implementing attention

$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix}\right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{Z} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

```
class SelfAttention(nn.Module):  
    def __init__(self, embed_dim):  
        super().__init__()  
        self.embed_dim = embed_dim  
  
        self.W_q = nn.Linear(embed_dim, embed_dim)  
        self.W_k = nn.Linear(embed_dim, embed_dim)  
        self.W_v = nn.Linear(embed_dim, embed_dim)  
  
    def forward(self, x):  
        Q = self.W_q(x)  
        K = self.W_k(x)  
        V = self.W_v(x)  
  
        scores = torch.matmul(Q, K.transpose(-2, -1))  
        scores = scores / (self.embed_dim ** 0.5)  
  
        weights = F.softmax(scores, dim=-1)  
        output = torch.matmul(weights, V)  
  
        return output, weights
```

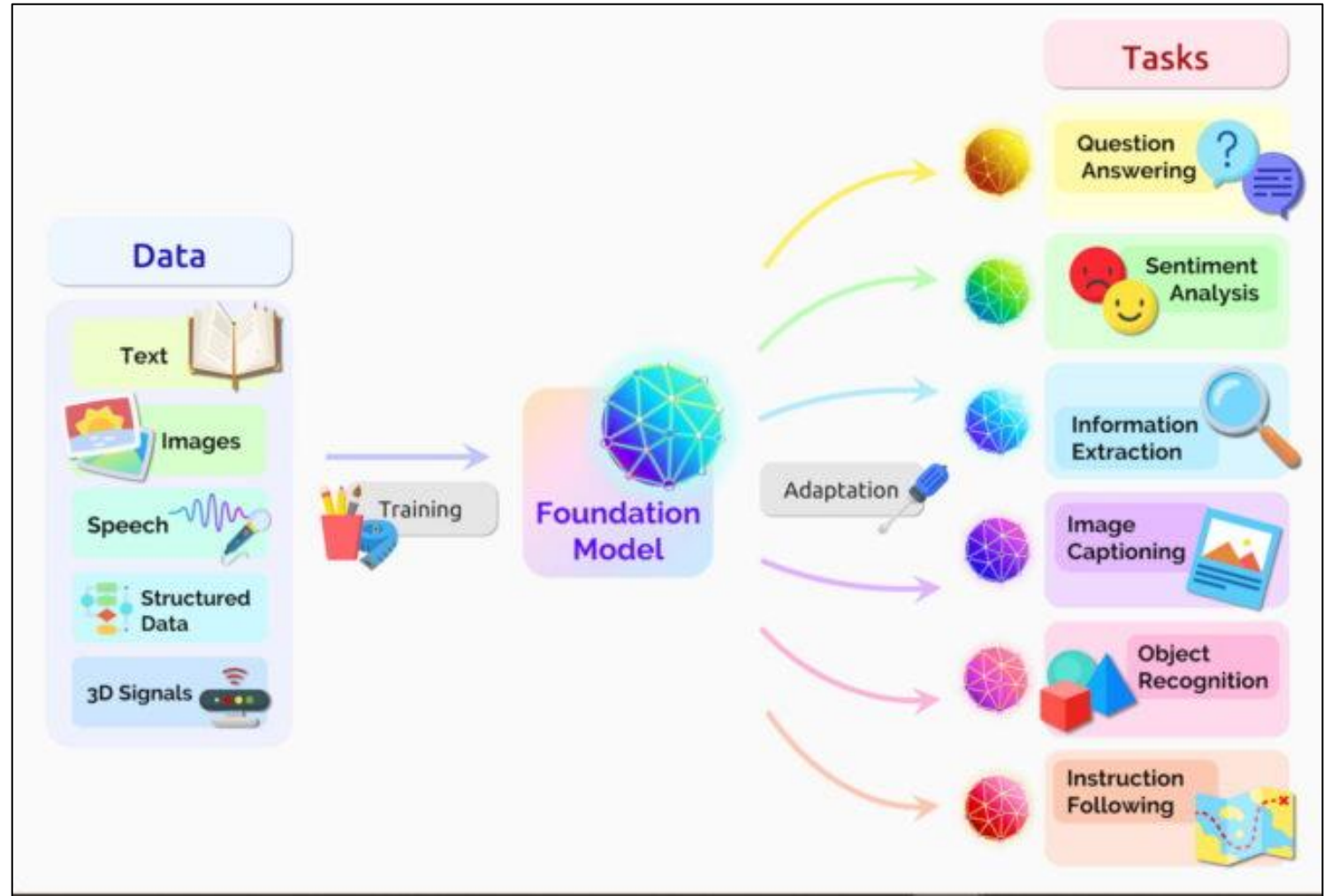
Foundation models

Foundation Models (FMs) for generalizing across tasks

Foundation models are **large** neural networks.

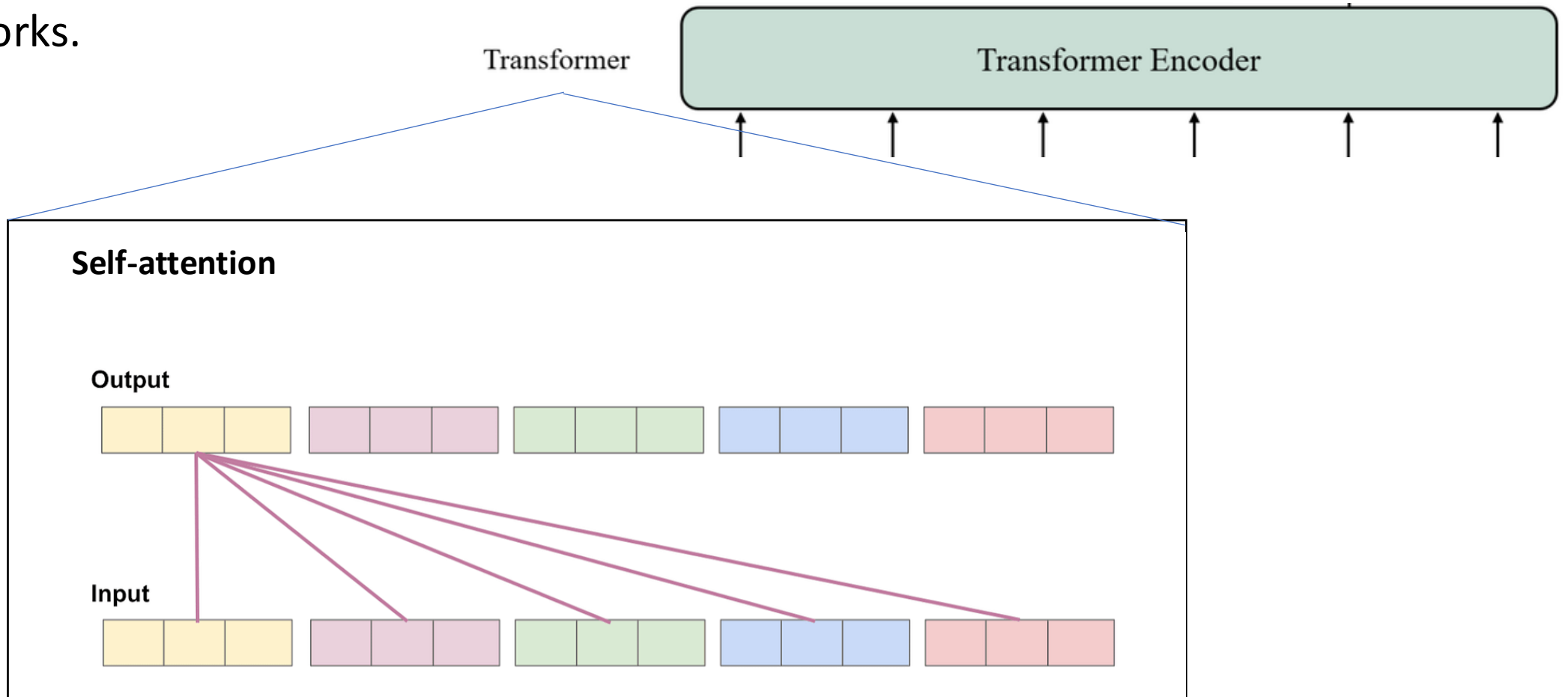
Trained on **large amounts** of **unlabeled** data (self-supervised learning).

Unlike more narrow traditional machine learning models, foundation models can be **easily adapted to many different tasks** without new training data.



FM architecture and training for language models

Foundation models are **large** neural networks.



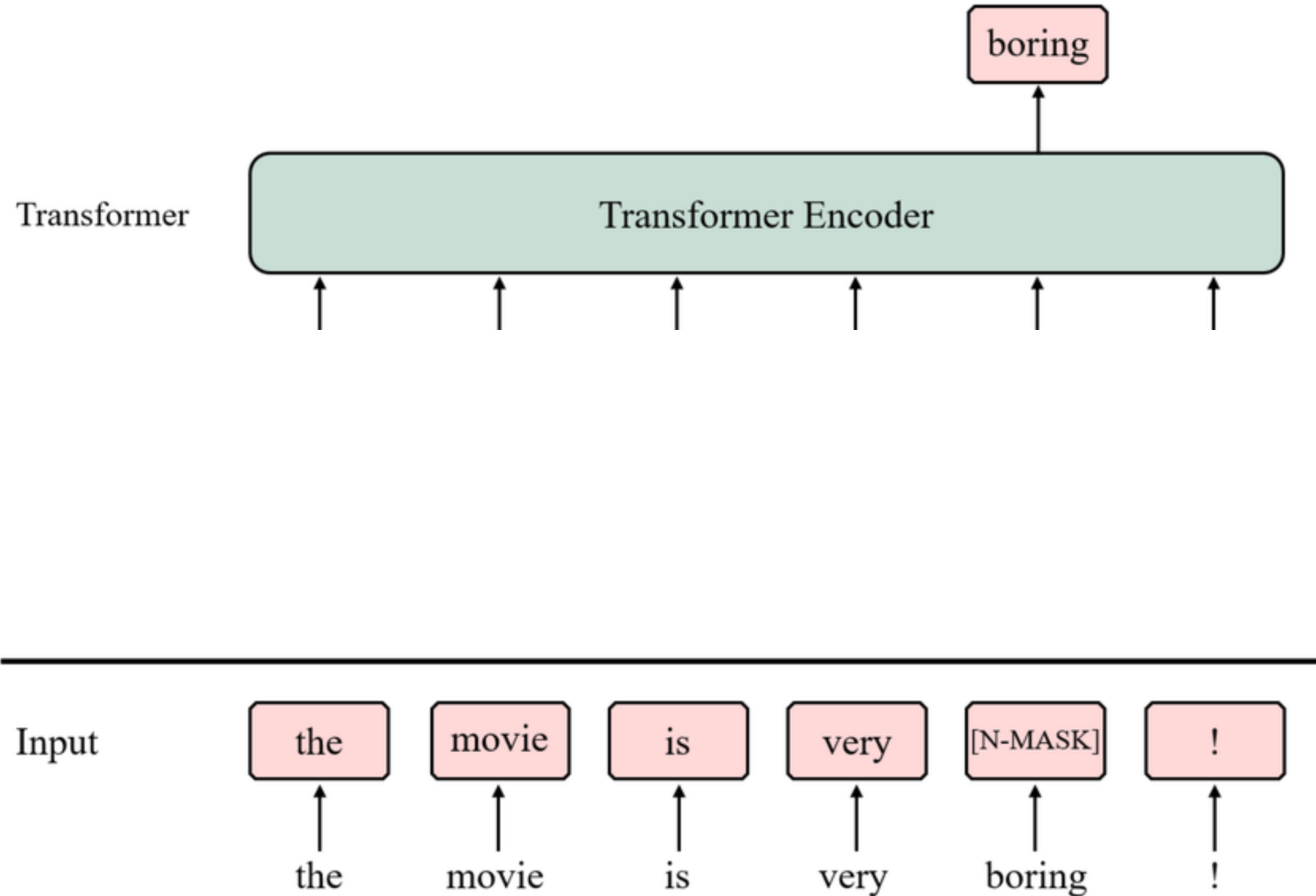


FM architecture and training for language models

Foundation models are **large** neural networks.

Trained on **large amounts** of **unlabeled** data (self-supervised learning).

Unlike more narrow traditional machine learning models, foundation models can be **easily adapted to many different tasks** without new training data.

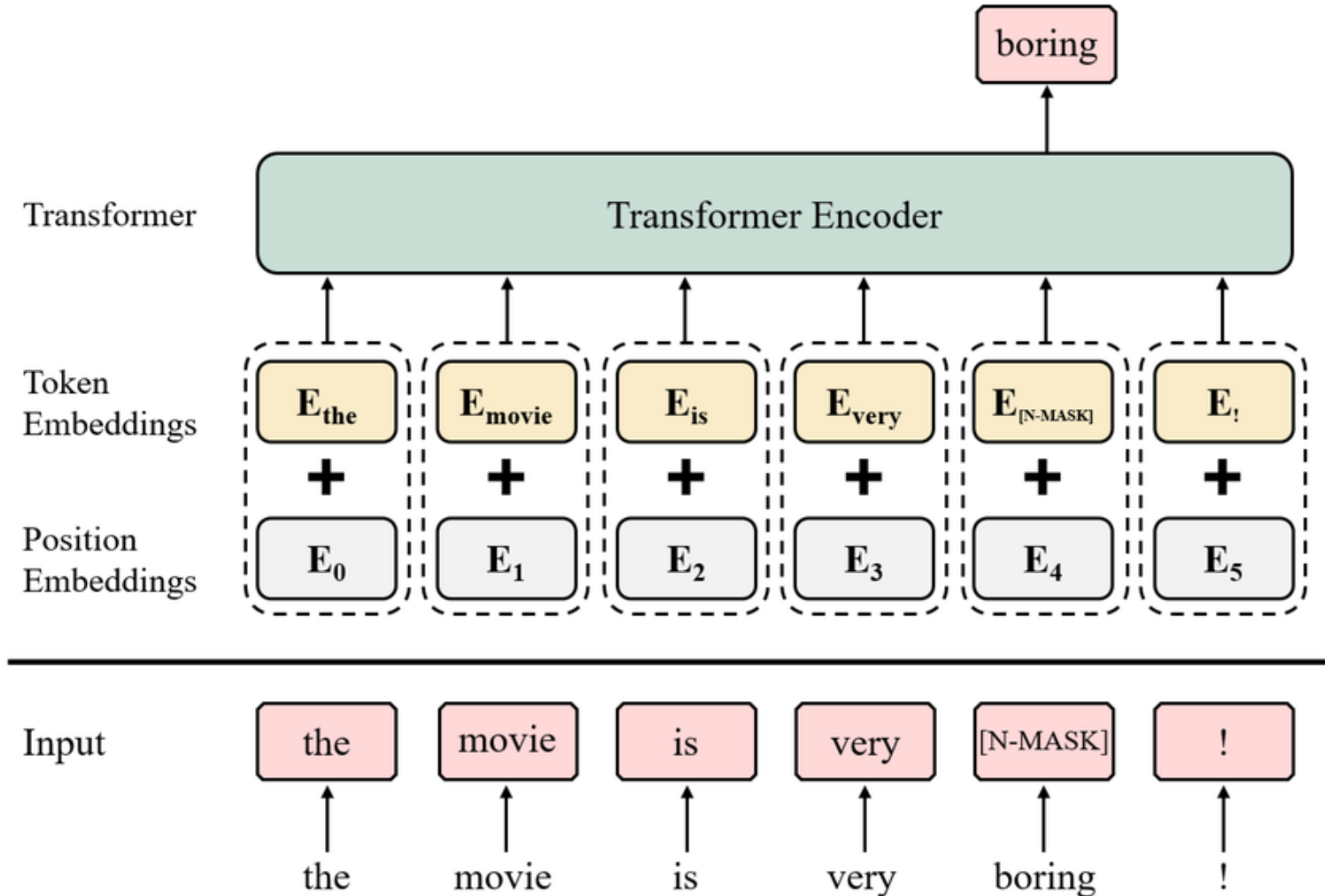


FM architecture and training for language models

Foundation models are **large** neural networks.

Trained on **large amounts** of **unlabeled** data (self-supervised learning).

Unlike more narrow traditional machine learning models, foundation models can be **easily adapted to many different tasks** without new training data.





Foundation Model Architectures

Encoder-Only



Bidirectional Attention

Sees entire input sequence at once

Training: Masked language modeling (MLM)

Examples: BERT, RoBERTa



Best For:

- Classification tasks
- Named entity recognition
- Question answering
- Embeddings & similarity

Decoder-Only



Causal/Autoregressive

Only sees previous tokens (left-to-right)

Training: Next-token prediction

Examples: GPT, Claude, LLaMA



Best For:

- Text generation
- Instruction following
- Dialogue & conversation
- Code generation

Note: Decoder-only models have become dominant for large foundation models due to better scalability

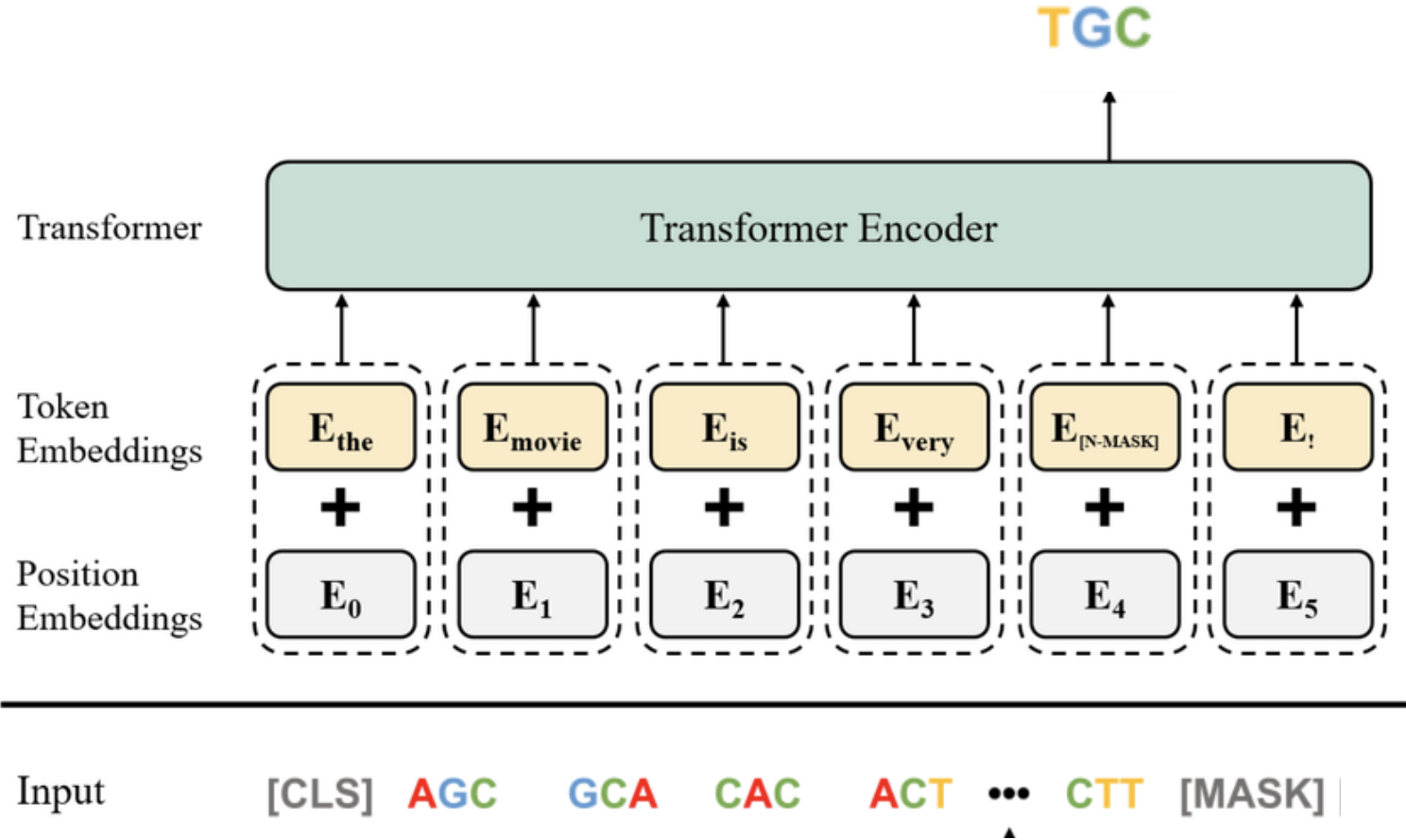
We now have FMs in genomics as well (DNA FMs)



DNABERT¹

DNABERT2²

Nucleotide Transformer³

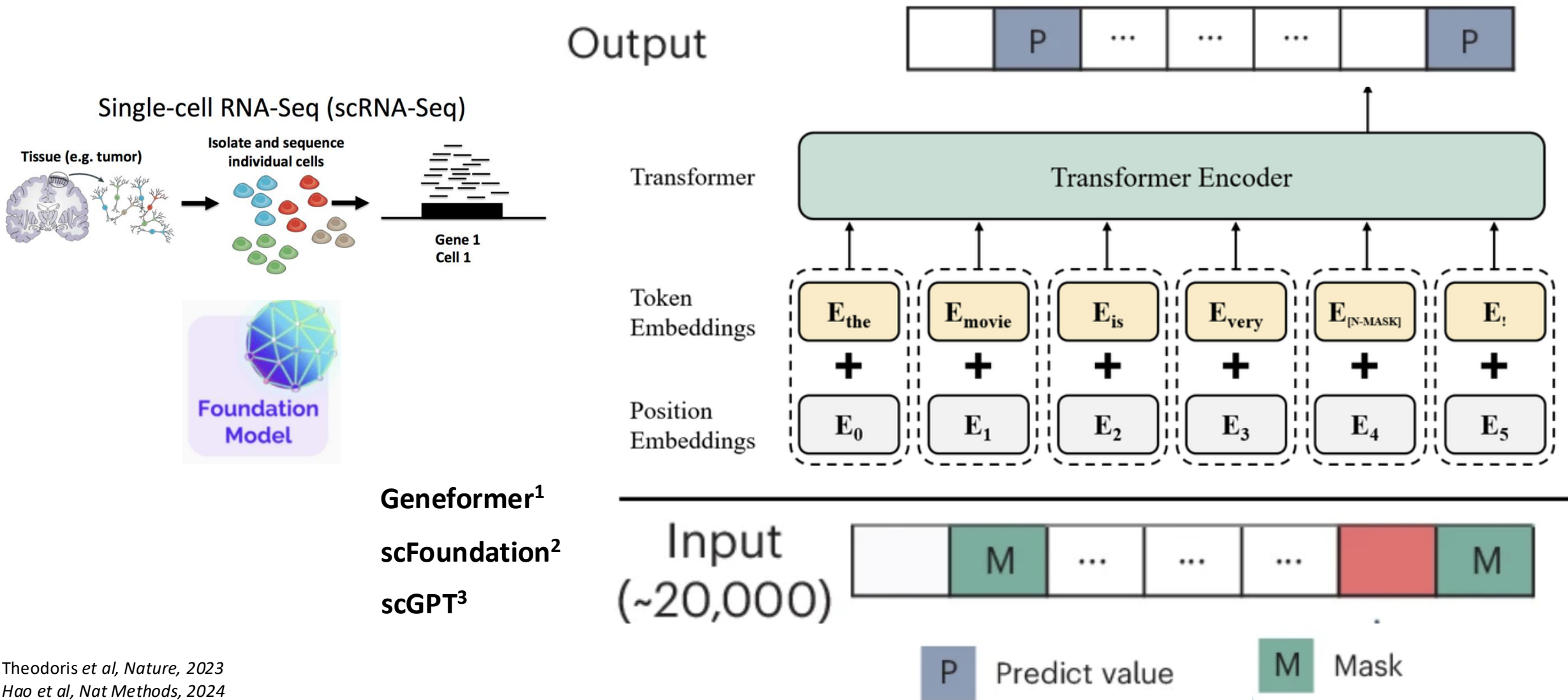


1. Ji et al, Bioinformatics, 2021

2. Zhou et al, ICML, 2024

3. Dalla-Torre et al, Nat Methods, 2024

We now have FMs in genomics as well (Single-cell FMs)



1. Theodoris et al, Nature, 2023
2. Hao et al, Nat Methods, 2024
3. Wang et al, Nat Methods, 2024
4. Chen et al, Nat Methods, 2025
5. Tejada-Lapuerta et al, Nat Methods, 2025

Recap– continue learning about Transformers

(1) Encoder: Positional encodings, Multi-headed attention, Modifications for efficiency

(2) Decoder

(3) Class activity: Implementing attention

(4) Foundation models

Wrap up



What was the clearest point today?

What was the muddiest point today?

